**UNIVERSIDAD CONTINENTAL**
**ESCUELA DE POSGRADO**
**MAESTRÍA EN CIENCIA DE DATOS**

Gestión de datos y bases de datos con SQL (PSMA00611)

# Base de datos para una aerolínea comercial en Estados Unidos: requisitos, modelado, implementación en SQL Server, integración con Python y preparación para Azure SQL Database

Producto Académico Colaborativo

Integrantes que participaron activamente: [POR COMPLETAR]

Docente: [POR COMPLETAR]

Sección / NRC: [POR COMPLETAR]

Líder del equipo (entrega en el aula virtual): [POR COMPLETAR]

Lugar y fecha: [POR COMPLETAR], noviembre de 2026

# Contenido

**Resumen ejecutivo** ..... **3**

**Problema, objetivos y alcance** ..... **4**

Problema ..... 4
Objetivos ..... 4
Alcance ..... 5

**Requisitos y reglas de negocio** ..... **6**

Fuentes y discrepancias ..... 6
Actores y casos de uso ..... 6
Decisiones de dominio ..... 6
Reglas verificables ..... 7

**Fundamentos conceptuales** ..... **8**

**Modelado y normalización** ..... **10**

Modelo conceptual ..... 10
Modelo lógico ..... 11
Normalización ..... 15
Desnormalizaciones controladas ..... 16

**Diccionario de datos** ..... **18**

**Arquitectura local y en la nube** ..... **20**

**Implementación SQL y Python** ..... **22**

Scripts T-SQL ..... 22
Reglas en el motor ..... 22
Control de concurrencia ..... 22
Procedimientos de negocio ..... 23
Aplicación de Python ..... 23

**Datos y metodología de validación** ..... **25**

Datos sintéticos reproducibles ..... 25
Niveles de prueba ..... 25

**Resultados y discusión** ..... **27**

Resultados de las pruebas ..... 27
Hallazgos del desarrollo ..... 28
Resultados de las consultas ..... 28
Discusión ..... 37

**Seguridad, limitaciones y trabajo futuro** ..... **38**

Seguridad ..... 38
Limitaciones ..... 38
Trabajo futuro ..... 38

**Conclusiones** ..... **40**

**Recomendaciones** ..... **41**

**Referencias** ..... **42**

**Anexos** ..... **44**

Anexo A. Matriz de trazabilidad ..... 44
Anexo B. Hallazgos corregidos durante el desarrollo ..... 46
Anexo C. Modelo físico del área de ventas ..... 48
Anexo D. Reproducción y archivos ..... 49
Anexo E. Declaración sobre el uso de herramientas y datos ..... 50

## Resumen ejecutivo

Este informe presenta el diseño, la implementación y la validación de una base de datos relacional para una aerolínea comercial nueva que operará vuelos domésticos en Estados Unidos. El encargo del curso pide una base de datos en la nube que cubra la definición de requisitos, el modelo de datos, la creación de tablas, la carga de datos, la definición de consultas y su ejecución desde un lenguaje de programación. El equipo añadió como metas propias la reproducibilidad completa, el control de concurrencia, la seguridad por roles y un conjunto de pruebas automatizadas.

La solución tiene 39 tablas de negocio en cinco esquemas temáticos, más un esquema de vistas de reporte, y aplica 70 restricciones `CHECK`, 53 claves foráneas, 29 restricciones de unicidad, 6 índices únicos filtrados y 12 triggers que garantizan en el propio motor 42 reglas de negocio: entre otras, que no haya sobreventa ni asientos duplicados, que el precio pactado de un tiquete no cambie aunque cambie la tarifa, que los pagos no superen el precio y que los reembolsos sigan la política de cancelación. Dieciséis procedimientos almacenados orquestan las operaciones críticas (emitir, pagar, cancelar, generar vuelos), otros cuatro automatizan las pruebas y nueve vistas concentran las fórmulas de análisis para que todas las consultas usen las mismas definiciones.

Una aplicación de consola en Python (pyodbc y pandas) despliega los scripts en orden, genera y carga datos sintéticos deterministas, ejecuta 20 consultas analíticas con parámetros enlazados, exporta los resultados a CSV, produce los gráficos, documenta el modelo desde el catálogo del motor y ejecuta las pruebas. Con una semilla fija se generan 3 588 vuelos, 232 096 personas, 323 718 tiquetes, 388 003 pagos y 18 182 reembolsos, que se cargan en una sola transacción con todas las reglas activas en unos 90 segundos.

La validación se ejecutó en un contenedor Linux con SQL Server 2022 Developer: 23 de 23 pruebas de reglas de negocio, 4 de 4 pruebas de concurrencia con sesiones simultáneas reales y 23 de 23 pruebas de la aplicación superadas; los controles de conciliación devolvieron cero diferencias, incluida la identidad de caja al centavo. Las mismas pruebas y consultas se repitieron con el nivel de compatibilidad de SQL Server 2016 con los mismos resultados. Durante el desarrollo las pruebas detectaron 20 defectos reales, documentados con su causa y corrección.

Dos resultados quedan **pendientes** y se declaran como tales: la ejecución en la laptop Windows del equipo y el despliegue en Azure SQL Database, que es el requisito de nube del encargo. Para ambos se entregan guías, scripts y listas de evidencias; ninguna prueba local se presenta como sustituto de la evidencia en la nube. Todos los datos analizados son sintéticos, por lo que las cifras validan el modelo y las consultas, pero no describen ningún mercado real.

# Problema, objetivos y alcance

## Problema

Una aerolínea nueva necesita registrar de forma confiable qué vuelos ofrece, quién compra y quién viaja, cuánto se cobró y cuánto se devolvió, y qué avión y qué tripulación operan cada vuelo. Los errores en ese registro tienen consecuencias visibles: un asiento vendido dos veces, un vuelo sobrevendido, un cobro duplicado o un reembolso mal calculado afectan a los pasajeros y a la contabilidad. Además, la información debe servir para tomar decisiones (ocupación, canales de venta, cancelaciones, utilización de la flota), lo que exige definiciones de cálculo consistentes.

El enunciado del curso enumera las relaciones mínimas —clientes, empleados, rutas, vuelos, localizaciones, tiquetes, aviones, cabinas, pagos y descuentos— y pide «ser exhaustivo con los atributos» para que la información pueda usarse después en la toma de decisiones (Universidad Continental, 2026). El enunciado deja abiertas varias ambigüedades que el diseño debía resolver de forma explícita, por ejemplo la diferencia entre pasajero y comprador, entre horario y vuelo fechado, o cómo se combinan descuentos y pagos parciales.

## Objetivos

**Objetivo general.** Diseñar, implementar y validar una base de datos relacional para la gestión comercial y operativa de una aerolínea, desplegable en SQL Server local y en Azure SQL Database, con consultas analíticas ejecutables desde Python y resultados reproducibles.

**Objetivos específicos.**

1. **OE1 · Requisitos.** Identificar los requisitos del negocio, resolver sus ambigüedades y expresarlos como reglas verificables.
2. **OE2 · Modelo.** Construir los modelos conceptual, lógico y físico en tercera forma normal y justificar cualquier desnormalización.
3. **OE3 · Integridad.** Implementar en T-SQL las tablas y las reglas, priorizando las restricciones declarativas, de modo que la base rechace los datos inválidos y resista operaciones concurrentes.
4. **OE4 · Datos y consultas.** Cargar datos sintéticos coherentes y reproducibles, y responder preguntas de negocio con al menos quince consultas documentadas, ejecutadas desde Python.
5. **OE5 · Validación y nube.** Verificar con pruebas automatizadas y controles de conciliación, y preparar el despliegue en Azure SQL Database.

## Alcance

**Dentro del alcance:** red doméstica de Estados Unidos con 13 aeropuertos y 22 rutas activas; dos cabinas (VIP y económica); tiquetes de un tramo; compras con varios tiquetes (familias, ida y vuelta); pagos múltiples con distintos métodos; un descuento por tiquete; cancelación y reembolso; flota con configuraciones de asientos; tripulación con roles; estados y auditoría de cambios.

**Fuera del alcance:** impuestos y tasas, gestión dinámica de ingresos, sobreventa deliberada (*overbooking*) y compensaciones, equipaje y abordaje, programa de millas, conexiones con tarifa combinada, mantenimiento de aeronaves, cumplimiento normativo completo y sistemas reales de distribución. Los costos de operación tampoco se registran; por eso el informe **no** calcula rentabilidad.

# Requisitos y reglas de negocio

## Fuentes y discrepancias

Se revisaron cuatro documentos: el enunciado del producto en el aula virtual, el sílabo y dos guías de aprendizaje (una general y otra identificada como «VIERNES»). Las guías añaden dos requisitos que no figuran en el texto del aula virtual: subir los scripts a GitHub y exponer de 10 a 15 diapositivas en 20 minutos con participación aleatoria de todos los integrantes. Se registraron doce discrepancias entre documentos; las más relevantes son la semana de entrega del avance (semana 2, 3 o 4 según el documento), un resultado de aprendizaje de otra asignatura copiado en ambas guías, años de edición incorrectos en dos libros de lectura y la ausencia de un número mínimo de páginas, que la rúbrica menciona pero ningún documento fija. No se asumió que la guía «VIERNES» fuera la vigente; cada discrepancia queda anotada con su tratamiento en el anexo y en `docs/00_fuentes/revision_fuentes.md`.

## Actores y casos de uso

Los actores son el cliente registrado, el comprador sin registro, el agente de ventas (mostrador de aeropuerto u oficina de ciudad), el planificador de operaciones, el analista y el administrador de la base de datos. Se definieron trece casos de uso, desde «registrar persona» y «emitir tiquete» hasta «cancelar vuelo», «asignar tripulación» y «consultar indicadores».

## Decisiones de dominio

Las ambigüedades del enunciado se resolvieron con decisiones explícitas, que son también las primeras preguntas de cualquier defensa:

**Tabla 1. Decisiones de dominio adoptadas**

| Ambigüedad | Decisión | Consecuencia en el modelo |
|---|---|---|
| Pasajero, comprador y cliente | Toda persona puede comprar o viajar; «cliente» es la persona con cuenta registrada | `ven.persona` (supertipo) y `ven.cliente` (subtipo 1:0..1) |
| Compra sin registro | Se registra la persona sin cuenta | El comprador no registrado es una `persona` sin fila en `cliente` |
| Ruta, horario y vuelo | La ruta es el par origen-destino; la programación fija número, hora y días; el vuelo es una salida fechada | `ops.ruta`, `ops.programacion`, `ops.programacion_dia`, `ops.vuelo` |
| Localizaciones físicas y web | Aeropuertos, oficinas y canales web son localizaciones; el aeropuerto es un subtipo | `ref.localizacion` y `ref.aeropuerto` con clave foránea compuesta |
| Capacidad y asientos | La capacidad es por vuelo y cabina; el asiento es opcional al reservar | `ops.vuelo_cabina` (inventario) y asiento validado contra el mapa del avión |
| Precio histórico | El tiquete conserva el precio y el porcentaje de descuento de la venta | Copia inmutable en `ven.tiquete`; tarifas versionadas por vigencia |
| Descuentos | Un solo descuento por tiquete: el de mayor | Sin acumulación; reglas de cabina, |

| Ambigüedad | Decisión | Consecuencia en el modelo |
|---|---|---|
| | porcentaje aplicable | vigencia, anticipación y registro |
| Pagos | Varios pagos por tiquete; estados pendiente, confirmado, rechazado y anulado | Emisión automática al completar el pago; vencimiento a las 72 h |
| Cancelación y reembolso | Política por cabina y antelación, regla de 24 horas y reembolso total si cancela la aerolínea | `ven.fn_politica_reembolso` y reembolsos ligados al pago |
| Estados | Transiciones válidas como datos | `ref.estado_*` y `ref.transicion_*` |
| Zonas horarias | Horas locales con desfase; transacciones en UTC | `datetimeoffset` en vuelos, `datetime2` en UTC en ventas |
| Asignación de avión y tripulación | Sin solapes y con tiempo mínimo en tierra; tripulación completa para despegar | Triggers con bloqueo del recurso |

La regla de 24 horas sigue la exigencia del Departamento de Transporte de Estados Unidos: para reservas hechas con al menos siete días de antelación, la aerolínea debe permitir mantenerlas sin pago o cancelarlas sin penalidad durante 24 horas (Customer Service Plan, 2026). Se modeló la segunda opción.

## Reglas verificables

Las reglas se numeraron de RN-01 a RN-42, y cada una indica el mecanismo que la garantiza: restricción declarativa (D), trigger (T), procedimiento (P) o verificación por consulta (A). La tabla siguiente resume la distribución; el detalle completo está en `docs/01_requisitos/analisis_requisitos.md`.

**Tabla 2. Reglas de negocio según el mecanismo que las garantiza**

| Mecanismo | Ejemplos | Reglas |
|---|---|---|
| Declarativo (D) | Origen distinto de destino; asiento único por vuelo (índice filtrado); cabina del tiquete existente en el vuelo (clave foránea compuesta); precio final calculado | 16 |
| Trigger (T) | Transiciones de estado; solapes de avión y tripulación; tarifa vigente; inventario; inmutabilidad del precio; pagos y reembolsos | 20 |
| Procedimiento (P) | Política de reembolso; saldo disponible con mensaje de negocio (respaldado por trigger) | 2 |
| Combinado (D + T, T + P) | Sin sobreventa (CHECK + trigger); descuentos (CHECK + trigger); pagos solo sobre reservas (trigger + procedimiento) | 3 |
| Verificación (A) | Inventario igual a tiquetes activos | 1 |
| **Total** | | **42** |

Trece supuestos (SUP-01 a SUP-13) completan lo que el enunciado no fija: por ejemplo, 72 horas como plazo para pagar una reserva, un tripulante de cabina por cada 50 asientos o que la base no impone los límites de horas de vuelo de la normativa aeronáutica, aunque una consulta los vigila.

Los requisitos no funcionales son: despliegue en Azure SQL Database (RNF-01), consultas desde Python (RNF-02), scripts en GitHub (RNF-03), reproducibilidad con semilla fija (RNF-04), conexiones cifradas sin secretos en el repositorio (RNF-05), compatibilidad desde SQL Server 2016 SP1 (RNF-06), control de concurrencia (RNF-07) y separación de privilegios (RNF-08).

## Fundamentos conceptuales

**Base de datos y modelo relacional.** Una base de datos es una colección organizada de información que un sistema gestor almacena, protege y pone a disposición de las aplicaciones (Microsoft, 2023). Codd (1970) propuso representar los datos como relaciones (tablas) de tuplas, manipuladas con operaciones de conjuntos e independientes de su almacenamiento físico. Las claves identifican cada fila; las claves foráneas expresan las relaciones entre tablas y permiten al motor rechazar referencias inexistentes. Elmasri y Navathe (2011) y Silberschatz et al. (2020) distinguen tres niveles de diseño: el conceptual (entidades y relaciones del negocio), el lógico (relaciones, claves y dependencias, independiente del producto) y el físico (tipos, índices y restricciones de un motor concreto). Este proyecto sigue esa secuencia.

**Normalización.** Una dependencia funcional X → Y indica que el valor de X determina el de Y. La primera forma normal exige atributos atómicos; la segunda elimina las dependencias parciales de una clave compuesta; la tercera elimina las dependencias transitivas entre atributos no clave (Elmasri & Navathe, 2011). El objetivo práctico es evitar anomalías de inserción, actualización y eliminación: que un dato se repita en muchas filas y pueda quedar inconsistente.

**Integridad declarativa y procedimental.** SQL permite expresar reglas como restricciones (`PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`, `NOT NULL`) que el motor aplica a cualquier cliente y en cualquier carga (Yaseen, 2019). Las reglas que involucran varias filas o tablas requieren triggers o procedimientos. Una restricción declarativa es preferible cuando basta, porque es más simple, más rápida y el optimizador la aprovecha. Los índices únicos filtrados de SQL Server permiten imponer unicidad solo sobre un subconjunto de filas, por ejemplo los tiquetes activos (Microsoft, s. f.-b).

**Transacciones y aislamiento.** Una transacción es una unidad atómica: o se aplican todos sus cambios o ninguno. El nivel de aislamiento determina qué ve una transacción de las demás. Berenson et al. (1995) mostraron que las definiciones del estándar no describen todas las anomalías posibles; entre ellas, la escritura sesgada (*write skew*): dos transacciones leen el mismo estado, cada una valida una regla y ambas escriben, de modo que el resultado conjunto viola la regla (Kleppmann, 2017). SQL Server ofrece lectura confirmada con versiones de fila (`READ_COMMITTED_SNAPSHOT`), predeterminada en Azure SQL Database, en la que las lecturas no bloquean a las escrituras (Microsoft, s. f.-e). Con ese nivel, las validaciones que deciden una escritura deben leer con bloqueo para no basarse en una versión antigua.

**Tiempo y zonas horarias.** Un instante se representa sin ambigüedad en UTC; la hora que ve el pasajero es local. El tipo `datetimeoffset` guarda ambas cosas y la función `AT TIME ZONE` convierte

entre zonas teniendo en cuenta el horario de verano, a partir de la lista de zonas que publica `sys.time_zone_info` (Microsoft, s. f.-a).

**Protección de datos personales.** El enmascaramiento dinámico de datos oculta parcialmente valores sensibles a los usuarios sin privilegio, sin cambiar los datos almacenados; no es cifrado y debe combinarse con permisos y auditoría (Microsoft, s. f.-d).

# Modelado y normalización

## Modelo conceptual

El modelo conceptual (figura 1) identifica las cosas del negocio que deben recordarse. Simplifica deliberadamente algunos detalles para leerse en una página: la relación aeropuerto-ruta aparece una vez aunque son dos (origen y destino) y la tripulación se muestra como relación muchos a muchos entre empleado y vuelo.

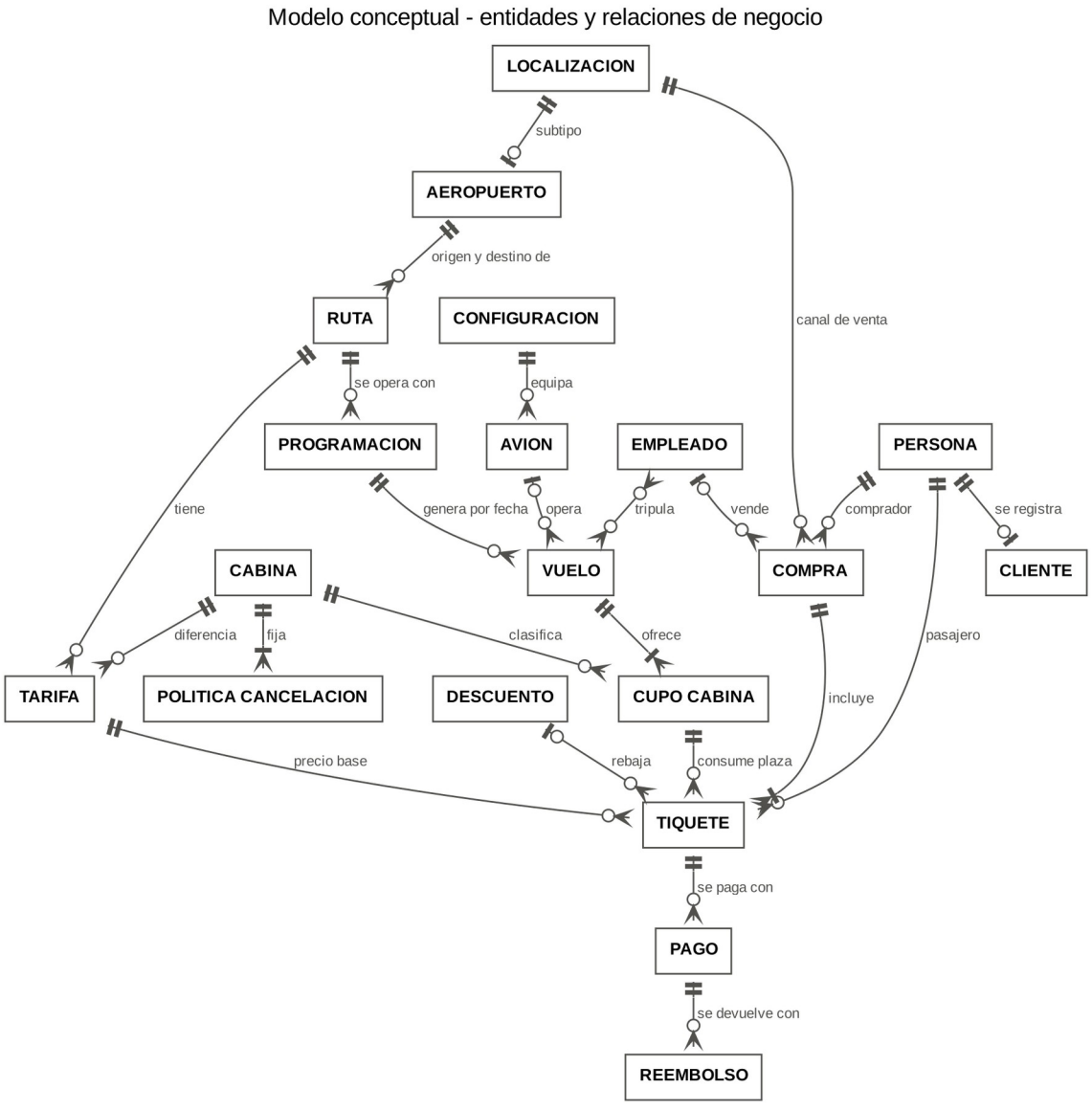



*Figura 1. Modelo conceptual. Fuente: `docs/02_modelo/diagramas/conceptual.mmd`.*

La entidad clave para el control de inventario es `CUPO_CABINA`: la capacidad de una cabina en un vuelo concreto. Separarla del vuelo permite controlar la venta por cabina y bloquear una sola fila al vender, como se explica en la sección de implementación.

## Modelo lógico

El modelo lógico se presenta en cuatro áreas temáticas (figuras 2 a 5) para que cada diagrama se lea a tamaño de página. Cada relación tiene clave primaria (PK); las claves foráneas (FK) implementan las relaciones del modelo conceptual, las claves únicas (UK) conservan las claves naturales y las entidades asociativas resuelven las relaciones muchos a muchos (asignación de tripulación, días de operación y asientos por configuración). Las entidades que aparecen solo con su clave pertenecen a otra área.

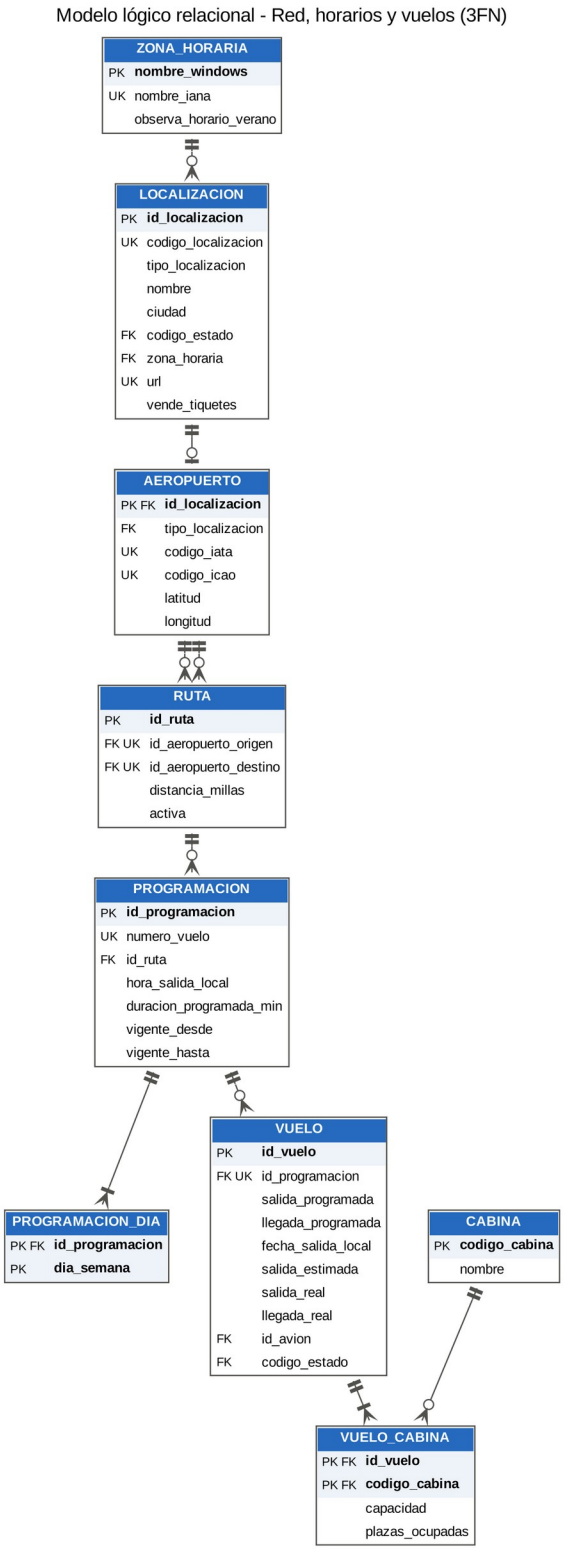



*Figura 2. Modelo lógico: red, horarios y vuelos. Fuente:* `docs/02_modelo/diagramas/logico_red_vuelos.mmd`.

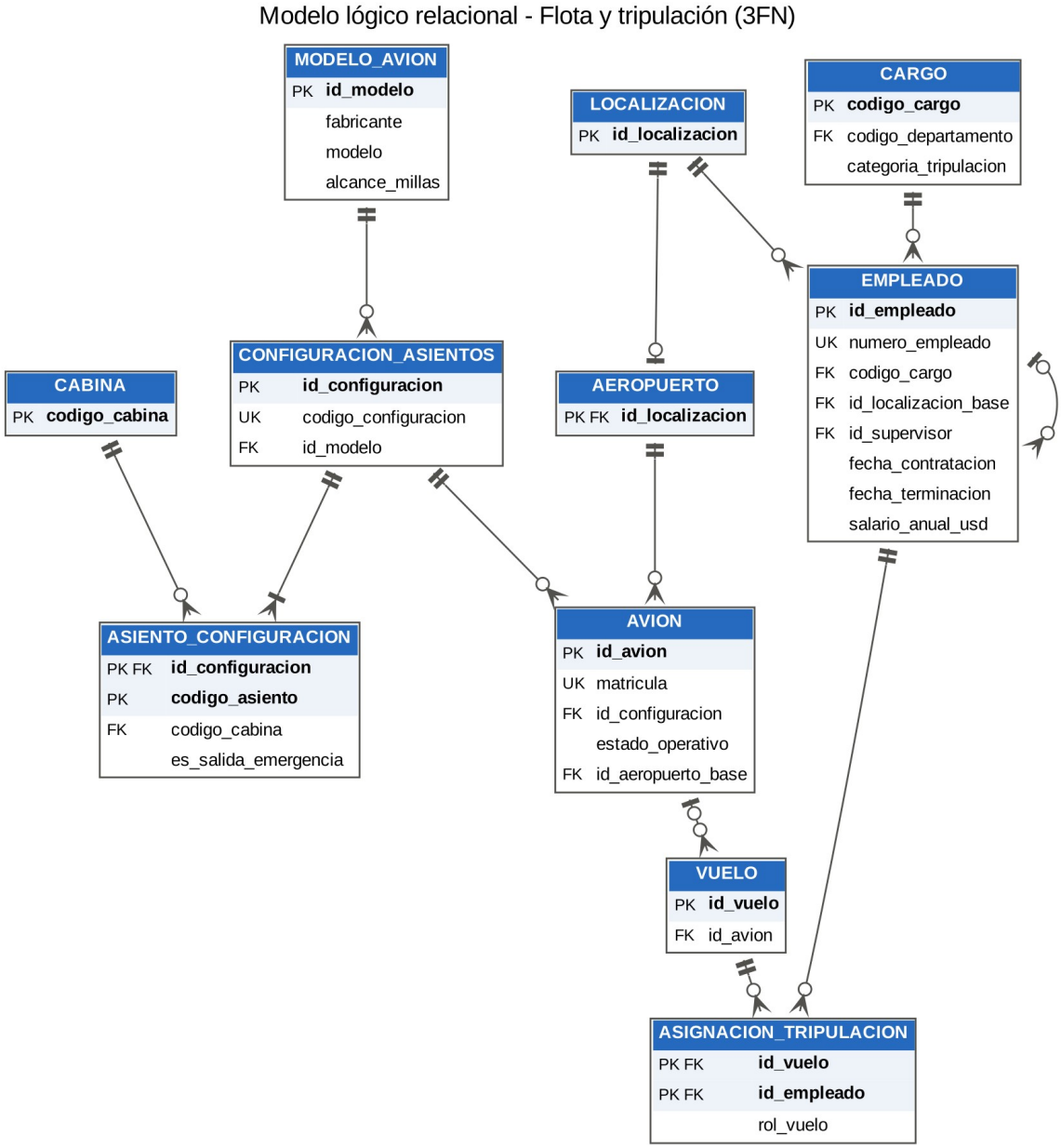



*Figura 3. Modelo lógico: flota y tripulación. Fuente:* `docs/02_modelo/diagramas/logico_flota_tripulacion.mmd`.

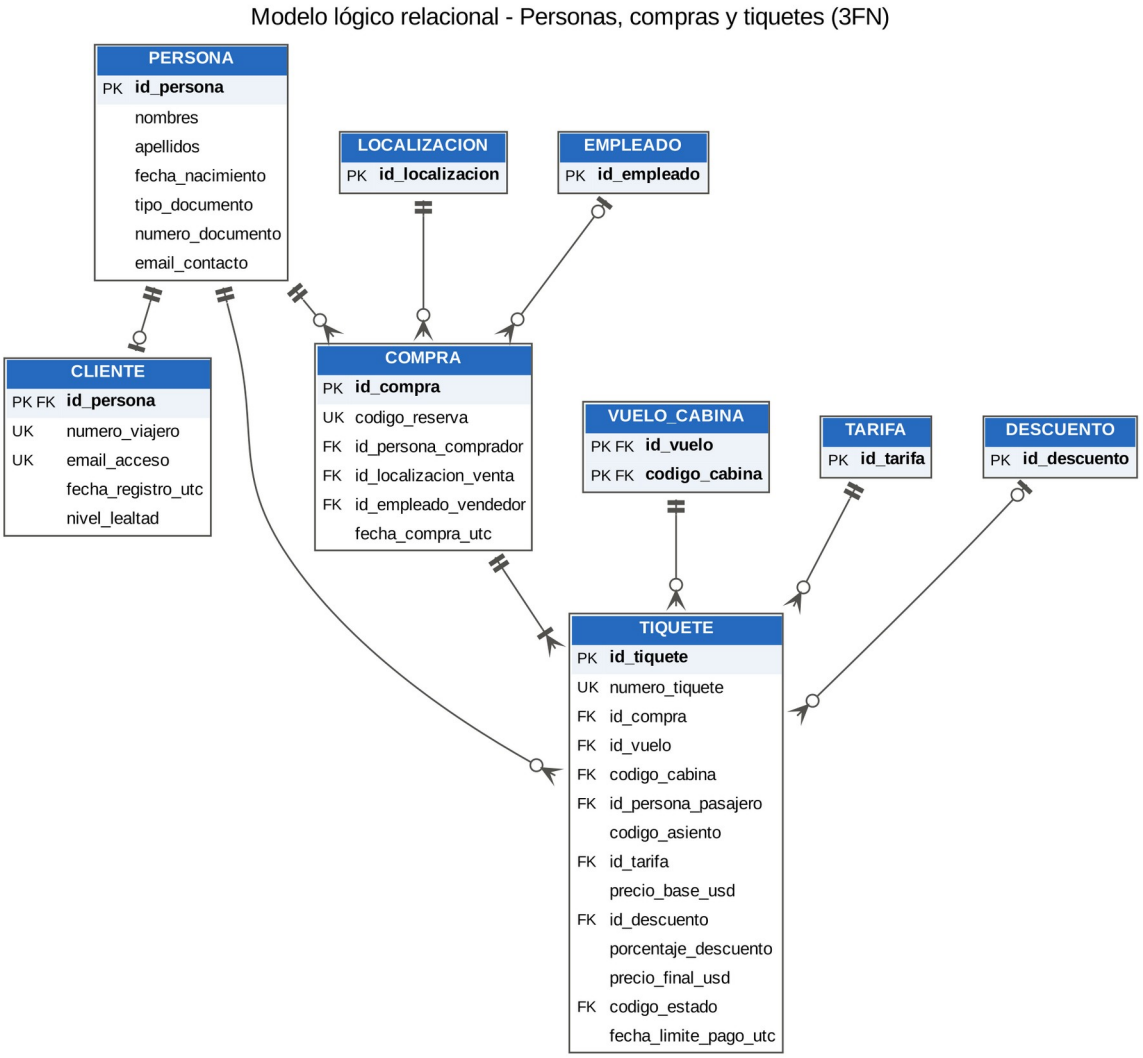



*Figura 4. Modelo lógico: personas, compras y tiquetes. Fuente:* `docs/02_modelo/diagramas/logico_personas_tiquetes.mmd`.

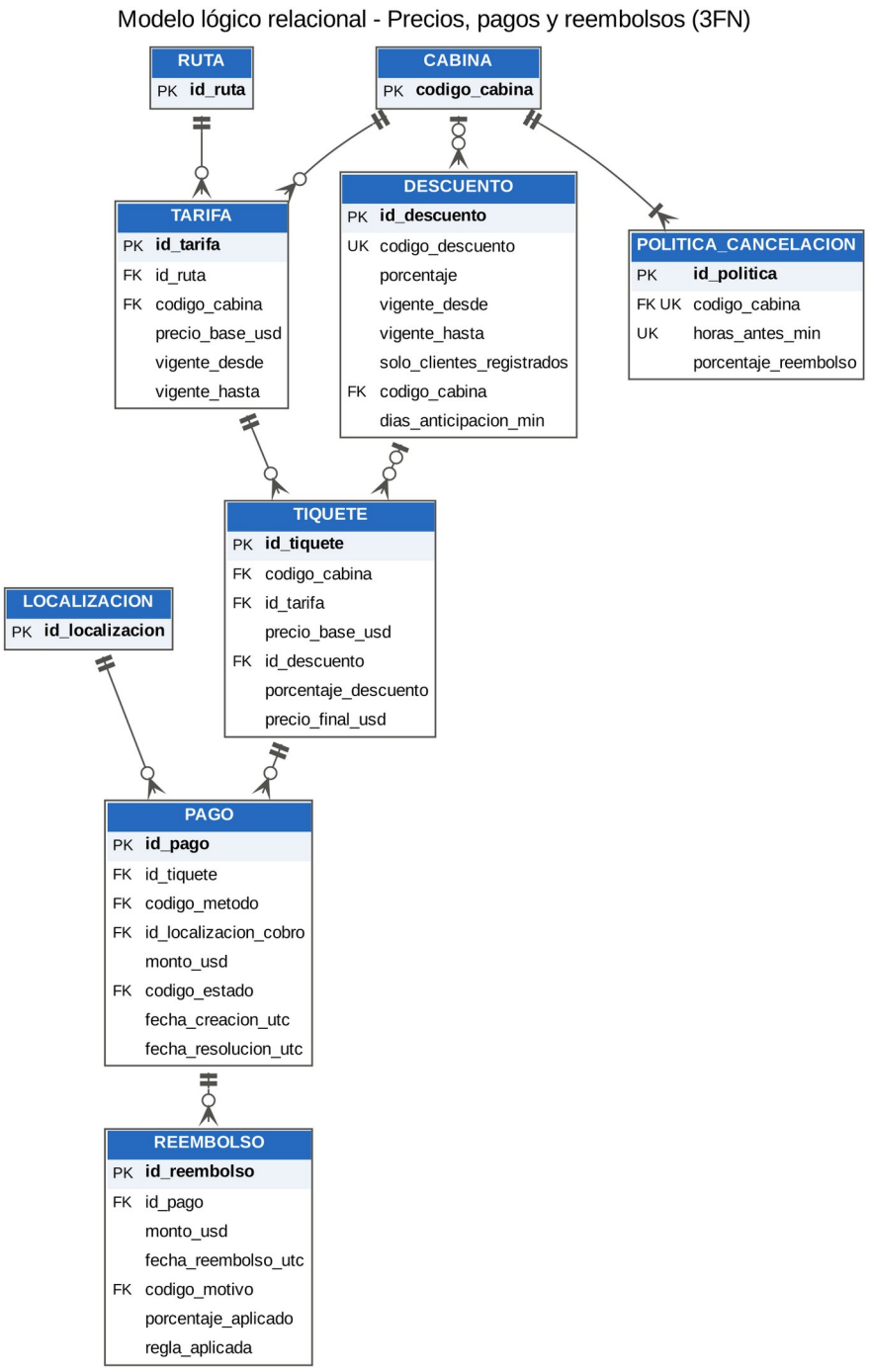



*Figura 5. Modelo lógico: precios, pagos y reembolsos. Fuente: `docs/02_modelo/diagramas/logico_precios_pagos.mmd`.*

## Normalización

Se partió de una «planilla de ventas» plana, con una fila por tiquete y columnas para el vuelo, el avión, el comprador, el pasajero, los pagos y los reembolsos. Esa estructura viola la primera forma normal (los pagos y los días de operación son listas) y produce todas las anomalías clásicas: no se

puede registrar un avión hasta venderle un tiquete, cambiar la hora de un vuelo exige modificar miles de filas y borrar los tiquetes de un vuelo cancelado elimina la información del avión.

La descomposición siguió las dependencias funcionales:

**Tabla 3. Descomposición de las dependencias transitivas de la planilla de ventas**

| Dependencia transitiva en la planilla | Relaciones resultantes |
|---|---|
| tiquete → vuelo → programación → ruta → aeropuertos | `ven.tiquete`, `ops.vuelo`, `ops.programacion`, `ops.ruta`, `ref.aeropuerto` |
| vuelo → avión → configuración → modelo | `ops.avion`, `ops.configuracion_asientos`, `ref.modelo_avion` |
| código de reserva → comprador, canal, vendedor, fecha | `ven.compra` (agrupa varios tiquetes) |
| documento → nombre de la persona | `ven.persona` |
| cargo → departamento, categoría de tripulación | `rh.cargo`, `rh.departamento` |
| estado → descripción, ocupa inventario, es final | `ref.estado_vuelo`, `ref.estado_tiquete`, `ref.estado_pago` |

Todas las relaciones con clave compuesta se revisaron para la segunda forma normal: en `ops.vuelo_cabina` la capacidad depende del vuelo **y** de la cabina, y en `ops.asignacion_tripulacion` el rol es del empleado **en ese vuelo**. La mayoría de las relaciones cumple además la forma normal de Boyce-Codd: sus únicos determinantes son claves candidatas. Las reglas de no solapamiento (vigencias de tarifas y de horarios) no son dependencias funcionales y se garantizan con triggers.

## Desnormalizaciones controladas

El diseño acepta seis redundancias, cada una con un mecanismo que impide la anomalía correspondiente:

**Tabla 4. Desnormalizaciones controladas y mecanismo que evita sus anomalías**

| Redundancia | Motivo | Control |
|---|---|---|
| Precio base y porcentaje de descuento copiados en el tiquete | El precio pactado es un hecho de la venta; si la tarifa cambia, el tiquete vendido no debe cambiar | El trigger verifica que la copia coincide con la tarifa y el descuento vigentes al vender; el precio del tiquete es inmutable y una tarifa usada no se edita: se crea otra versión |
| Contador de plazas ocupadas por vuelo y cabina | Evitar contar tiquetes en cada venta y disponer de una fila que bloquear ante ventas simultáneas | Lo mantiene el trigger en la misma transacción; un `CHECK` impide superar la capacidad; una vista y una prueba concilian el contador con el recuento real |
| Capacidad por vuelo y cabina | El vuelo se vende antes de asignarle avión | Se sincroniza al asignar avión; se prohíbe cambiar a una configuración distinta con ventas activas |
| Fecha local de salida del vuelo | Clave natural «XX101 del 6 de noviembre» | Columna calculada persistida, no editable |
| Tipo de localización constante en aeropuerto | Garantizar que el subtipo solo apunte a localizaciones de tipo aeropuerto | Columna calculada y clave foránea compuesta |
| Cabina en el tiquete junto a la tarifa | Dos claves foráneas compuestas: la cabina existe en el vuelo y la tarifa es de esa cabina | Restricciones declarativas |

Una decisión relacionada es **no** declarar una clave foránea del asiento del tiquete hacia el mapa de asientos: el mapa depende del avión asignado al vuelo, de modo que guardar la configuración en el tiquete crearía una dependencia transitiva que quedaría desactualizada al cambiar de avión. La existencia del asiento en la cabina correcta se valida con un trigger.

## Diccionario de datos

El diccionario se genera desde el catálogo del motor con `python -m aerolinea documentar`: tipos, longitudes, nulabilidad, valores por defecto, claves, restricciones `CHECK`, columnas calculadas y enmascaradas se leen de las vistas del sistema, y las definiciones de negocio se toman de un archivo mantenido por el equipo (`docs/02_modelo/definiciones_atributos.csv`). Así el diccionario coincide con el código por construcción. El resultado cubre 41 tablas y 269 columnas, todas con definición; se publica en Markdown, CSV y Excel, y como propiedades extendidas `MS_Description` visibles en SSMS (`sql/13_descripciones.sql`).

La tabla siguiente muestra, como ejemplo, la tabla central del modelo; el diccionario completo está en `docs/02_modelo/diccionario_datos.md` y el diagrama físico de ventas en el anexo C.

**Tabla 5. Diccionario de datos de la tabla `ven.tiquete`**

| Columna | Tipo | Nulo | Claves | Definición |
|---|---|---|---|---|
| id_tiquete | int | No | PK | Identificador del tiquete |
| numero_tiquete | char(13) | No | UQ uq_tiquete_numero | Número del documento de transporte |
| id_compra | int | No | FK → ven.compra | Compra a la que pertenece |
| id_vuelo | int | No | UX ux_tiquete_vuelo_asiento (filtrado); UX ux_tiquete_vuelo_pasajero (filtrado); FK → ops.vuelo_cabina | Vuelo |
| codigo_cabina | char(3) | No | FK → ven.tarifa; FK → ops.vuelo_cabina | Cabina |
| id_persona_pasajero | int | No | UX ux_tiquete_vuelo_pasajero (filtrado); FK → ven.persona | Pasajero (registrado o no) |
| codigo_asiento | varchar(4) | Sí | UX ux_tiquete_vuelo_asiento (filtrado) | Asiento asignado (opcional al comprar) |
| id_tarifa | int | No | FK → ven.tarifa | Versión de tarifa aplicada |
| precio_base_usd | decimal(10,2) | No | — | Precio base pactado en la venta (hecho histórico) |
| id_descuento | smallint | Sí | FK → ven.descuento | Descuento aplicado (como máximo uno) |
| porcentaje_descuento | decimal(5,2) | No | — | Porcentaje aplicado en la venta (copia histórica) |
| monto_descuento_usd | decimal(10,2) | Sí | — | Importe descontado |
| precio_final_usd | decimal(10,2) | Sí | — | Importe a pagar |
| codigo_estado | varchar(15) | No | FK → ref.estado_tiquete | Estado del tiquete |
| fecha_limite_pago_utc | datetime2(0) | No | — | Vencimiento de la reserva |
| fecha_emision_utc | datetime2(0) | Sí | — | Instante en que quedó pagado |
| fecha_cancelacion_utc | datetime2(0) | Sí | — | Instante de cancelación |
| codigo_motivo_cancelacion | varchar(15) | Sí | FK → ref.motivo_cancelacion | Causa de la cancelación |
| creado_utc | datetime2(0) | No | — | Instante de la venta del tiquete |
| version_fila | timestamp | No | — | Versión de fila para control de concurrencia |

| Columna | Tipo | Nulo | Claves | Definición |
|---|---|---|---|---|
| | | | | optimista |

*Fuente:* `docs/02_modelo/diccionario_datos.csv`, *generado desde el catálogo del motor.*

Convenciones de nombres: tablas y columnas en español, en minúsculas y con guion bajo; claves sustitutas `id_*`; restricciones con prefijo por tipo (`pk_`, `fk_`, `uq_`, `ck_`, `df_`, `ix_`, `ux_`); importes en `decimal(10,2)` con sufijo `_usd`; instantes transaccionales en `datetime2(0)` con sufijo `_utc`.

## Arquitectura local y en la nube

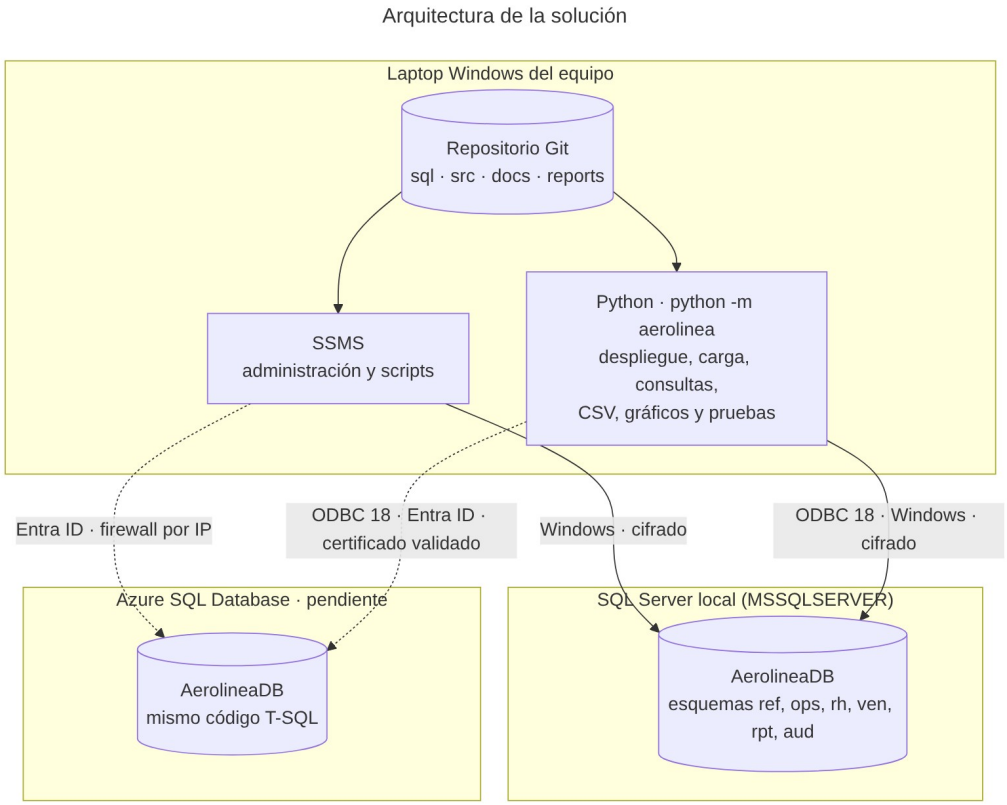



*Figura 6. Arquitectura de la solución. Fuente:* `docs/03_arquitectura/arquitectura.mmd`.

La figura 6 resume los componentes y las conexiones. La base se organiza en esquemas temáticos: `ref` (catálogos), `ops` (operación), `rh` (personal), `ven` (ventas), `rpt` (vistas de reporte), `aud` (auditoría), `adm` (registro de despliegue), `stg` (área de carga) y `tst` (pruebas). La separación permite conceder permisos por área —por ejemplo, el analista solo lee `rpt`— y deja visible la frontera entre datos operativos y analíticos.

**Entorno local.** El equipo dispone de una laptop Windows con SQL Server (instancia predeterminada `MSSQLSERVER`, en `localhost`) y usará autenticación de Windows. SSMS se usa para administrar y ejecutar scripts; el motor guarda los datos y garantiza las reglas; Python ejecuta las tareas reproducibles. La conexión local por memoria compartida no requiere habilitar TCP/IP, SQL Server Browser ni SQL Server Agent. El controlador ODBC 18 cifra la conexión por defecto; como la instancia local usa un certificado autofirmado, en desarrollo se acepta ese certificado (`TrustServerCertificate=yes`) solo para `localhost`, manteniendo el cifrado. La verificación de versión, edición, permisos, Python y controlador está automatizada en un script de solo lectura y **queda pendiente de ejecutar en la laptop**.

**Entorno de nube.** El mismo código T-SQL se despliega en Azure SQL Database. Las diferencias relevantes se resolvieron en el diseño: los scripts no usan `USE` (en Azure no cambia de base), la base se crea desde el portal o la CLI, la autenticación es de Microsoft Entra ID, el certificado del servidor se valida siempre y `READ_COMMITTED_SNAPSHOT` está activo por defecto, igual que en la configuración local elegida. Se propone la oferta gratuita de Azure SQL Database (General Purpose serverless con un límite mensual de cómputo y pausa automática al agotarlo) para no generar costos (Microsoft, s. f.-c), con acceso restringido por firewall a la IP del equipo. El script `scripts/azure/crear_recursos_azure.ps1` solo muestra los comandos salvo que se confirme explícitamente, y `sql/azure/verificacion_azure.sql` reúne las evidencias de solo lectura. **No se creó ningún recurso**: el despliegue requiere una suscripción y la autorización del equipo.

**Compatibilidad.** El código evita funciones posteriores a SQL Server 2016 SP1 (por ejemplo `STRING_AGG`, `TRIM` o `GREATEST`). Se ejecutó en SQL Server 2022 con el nivel de compatibilidad 160 y, de nuevo, con el nivel 130 (SQL Server 2016), con los mismos resultados; no se probó en un motor 2016, 2017 o 2019 real.

# Implementación SQL y Python

## Scripts T-SQL

La base se construye con scripts numerados, idempotentes (pueden ejecutarse de nuevo sin error) y protegidos contra la ejecución accidental en bases del sistema:

**Tabla 6. Scripts T-SQL del despliegue**

| Script | Contenido |
|---|---|
| 01 | Creación de la base local con intercalación `SQL_Latin1_General_CP1_CI_AS` y `READ_COMMITTED_SNAPSHOT` |
| 02-04 | Esquemas, secuencias, 41 tablas, restricciones e índices |
| 05 | Funciones (calendario ISO, política de reembolso) y 12 triggers de reglas de negocio |
| 06-07 | 9 vistas de reporte y 16 procedimientos almacenados de negocio |
| 08 | Roles, permisos y enmascaramiento dinámico |
| 09-10 | Datos de referencia y procedimiento de carga sintética |
| 11 | Ejemplos de `INSERT`, `UPDATE` y `DELETE` seguros (fuera del despliegue) |
| 12-13 | Pruebas automatizadas (4 procedimientos del esquema `tst`) y descripciones `MS_Description` |
| 99 | Eliminación de la base local: separado, con confirmación explícita y rechazo de Azure |

`GO` no es una instrucción del motor sino un separador que interpretan SSMS y `sqlcmd`; la aplicación divide los lotes del lado del cliente respetando comentarios anidados y cadenas de texto.

## Reglas en el motor

La mayoría de las reglas se expresa con restricciones declarativas. Algunos ejemplos representativos:

```
-- Un asiento no se vende dos veces en el mismo vuelo (solo cuentan los
tiquetes activos).
CREATE UNIQUE INDEX ux_tiquete_vuelo_asiento ON ven.tiquete (id_vuelo,
codigo_asiento)
WHERE codigo_asiento IS NOT NULL
  AND codigo_estado IN ('RESERVADO', 'EMITIDO', 'USADO', 'NO_PRESENTADO');

-- La cabina del tiquete existe en el vuelo y la tarifa es de esa misma
cabina.
ALTER TABLE ven.tiquete ADD
  CONSTRAINT fk_tiquete_vuelo_cabina FOREIGN KEY (id_vuelo, codigo_cabina)
      REFERENCES ops.vuelo_cabina (id_vuelo, codigo_cabina),
  CONSTRAINT fk_tiquete_tarifa_cabina FOREIGN KEY (id_tarifa, codigo_cabina)
      REFERENCES ven.tarifa (id_tarifa, codigo_cabina);
```

Las reglas entre filas o tablas usan triggers escritos en forma de conjuntos: validan todas las filas de `inserted` y `deleted`, de modo que funcionan igual con un tiquete que con una carga de 300 000. Ante una violación lanzan un error numerado (50001-50069) con un mensaje que cita la regla; los procedimientos usan la serie 501xx y traducen los errores del motor a mensajes de negocio. Los estados válidos y sus transiciones son datos (`ref.transicion_*`), no listas fijas en el código.

## Control de concurrencia

Cuatro mecanismos impiden las anomalías de las operaciones simultáneas:

1. **Inventario como semáforo.** Al vender, el trigger lee la fila de `ops.vuelo_cabina` con bloqueo de actualización (`UPDLOCK`): la segunda venta del mismo vuelo y cabina espera a que la primera termine, mientras que las ventas de otros vuelos no se bloquean. Una restricción `CHECK (plazas_ocupadas <= capacidad)` es la última barrera.
2. **Asiento único.** El índice único filtrado garantiza que solo una transacción obtenga un asiento; la otra recibe un error que el procedimiento traduce a «asiento ocupado».
3. **Pagos serializados por tiquete.** El procedimiento y el trigger de pagos bloquean la fila del tiquete antes de sumar lo cobrado.
4. **Solapes de avión y tripulación.** Se bloquea el recurso (avión o empleado) y se revisa su agenda ordenada con la función `LAG`.

Se descartaron `SERIALIZABLE` en todas las transacciones (bloqueos amplios e interbloqueos), `SNAPSHOT` (no evita la escritura sesgada entre filas distintas) y los bloqueos de aplicación (quedan fuera del modelo de datos).

## Procedimientos de negocio

`ven.usp_emitir_tiquete` busca la tarifa vigente en la fecha de venta, valida y aplica el descuento, calcula el plazo de pago (el menor entre 72 horas y tres horas antes de la salida), genera el código de reserva y el número de tiquete, y deja el tiquete en `RESERVADO`. `ven.usp_registrar_pago` bloquea el tiquete, verifica el saldo y, al completarse el pago, emite el tiquete. `ven.usp_cancelar_tiquete` aplica la política mediante `ven.fn_politica_reembolso`, anula los pagos pendientes y reparte el reembolso entre los pagos confirmados. `ops.usp_generar_vuelos` crea los vuelos fechados a partir de la programación semanal, convirtiendo la hora local de salida a la zona del aeropuerto de origen y la llegada a la del destino; es idempotente. Todos usan `SET XACT_ABORT ON`, `TRY/CATCH` y transacción propia solo si no hay una abierta.

## Aplicación de Python

La aplicación (`src/aerolinea`) se ejecuta con `python -m aerolinea <comando>`. Lee la configuración de un archivo `.env` o de variables de entorno, elige el controlador ODBC 18 o 17, admite autenticación de Windows, SQL o Microsoft Entra ID y rechaza configuraciones inseguras para Azure (sin cifrado o sin validar el certificado). Las consultas se ejecutan siempre con parámetros enlazados: los valores de ejemplo de cada consulta se sustituyen por marcadores `?` y se envían por separado, de modo que la entrada del usuario nunca se concatena en el texto SQL. Los procedimientos con parámetros de salida se invocan con un lote `DECLARE … EXEC … SELECT`, porque pyodbc no los admite de forma directa.

Tres detalles de implementación resultaron decisivos y se descubrieron con pruebas: recorrer todos los conjuntos de resultados con `nextset()` para que un error de una sentencia posterior del lote no pase inadvertido; convertir el tipo `datetimeoffset`, que pyodbc no reconoce; y desactivar el *pooling* de conexiones del administrador de controladores, que en Linux reutilizaba sesiones sin restablecerlas y trasladaba una transacción abierta de una conexión a la siguiente (hallazgo H-13).

# Datos y metodología de validación

## Datos sintéticos reproducibles

Los datos de referencia (13 aeropuertos, 19 localizaciones, 24 rutas, 32 programaciones, 17 aviones con 586 asientos en 4 configuraciones, 96 tarifas, 6 descuentos y 353 empleados) se insertan con T-SQL. Los datos transaccionales los genera `generador.py` con una semilla fija (20261106) y una fecha de corte (1 de noviembre de 2026, 00:00 UTC): ventas desde el 1 de julio y operación desde el 1 de septiembre hasta el 31 de diciembre de 2026.

**Tabla 7. Volumen de datos sintéticos cargados**

| Tabla | Filas |
|---|---|
| Vuelos (`ops.vuelo`) | 3 588 |
| Asignaciones de tripulación | 20 064 |
| Personas / clientes registrados | 232 096 / 30 000 |
| Compras | 168 928 |
| Tiquetes | 323 718 |
| Pagos | 388 003 |
| Reembolsos | 18 182 |
| Cambios de estado auditados | 1 248 064 |

El generador incluye a propósito casos difíciles: compras sin registro, viajes de ida y vuelta, pagos en dos o tres partes, pagos rechazados, reservas que vencen, cancelaciones dentro de las 24 horas, cancelaciones de vuelos, demoras propagadas por avión, no presentados, mantenimiento programado con avión de reserva, cambio de horario del 1 de noviembre y llegadas al día siguiente. Los archivos CSV generados tienen una huella SHA-256 registrada en un manifiesto, de modo que cualquier ejecución con la misma semilla puede comprobarse byte a byte.

La carga pasa por tablas de staging y se aplica en **una sola transacción con todas las restricciones y triggers activos**. Si un dato viola una regla, la transacción entera se revierte. Que la carga termine es, por tanto, una prueba de integración: las casi 1,2 millones de filas cargadas cumplen todas las reglas (además, los triggers registran 1,25 millones de cambios de estado en la auditoría).

## Niveles de prueba

**Tabla 8. Niveles de prueba**

| Nivel | Qué verifica | Herramienta |
|---|---|---|
| Aplicación | División de lotes, configuración segura, escape de credenciales, traducción de errores, documentación de consultas, política de reembolso | pytest (23 pruebas) |
| Reglas de la base | Cada regla crítica se intenta violar y se comprueba el rechazo con el número de error esperado; los casos válidos se ejecutan de punta a punta | `tst.usp_ejecutar_pruebas` (PR-01 a PR-23) |
| Concurrencia | Sobreventa, asiento duplicado, doble cobro y asignación automática de asiento con sesiones simultáneas reales | Hilos con una conexión cada uno y una barrera de arranque (PC-01 a PC-04) |

| Nivel | Qué verifica | Herramienta |
|---|---|---|
| Integración | Que los datos sintéticos cumplan todas las reglas | Carga transaccional |
| Consultas | Que las consultas midan lo que dicen | Controles de conciliación y recuperación de parámetros conocidos del generador |

Una prueba se declara superada solo si se observó su resultado. Las pruebas crean sus escenarios en vuelos reservados para ese fin y los eliminan al terminar, sin alterar los datos analizados.

**Validación por parámetros conocidos.** Como el generador fija algunas probabilidades (42 % de compradores registrados, 5 % de cancelaciones voluntarias, 4,5 % de no presentados), una consulta correcta debe recuperar valores cercanos. Esto valida las consultas, pero no dice nada sobre el mundo real.

# Resultados y discusión

## Resultados de las pruebas

Todas las pruebas ejecutadas en el contenedor de validación se superaron. La tabla resume cada caso; el detalle observado está en `evidence/contenedor_linux/resultados_pruebas.md`.

**Tabla 9. Resultados de las pruebas de reglas y de concurrencia (contenedor de validación)**

| ID | Tipo | Regla | Caso probado | Resultado |
|---|---|---|---|---|
| PR-01 | T-SQL | FK | Un pago de un tiquete inexistente es rechazado (clave foránea) | Superada |
| PR-02 | T-SQL | CHECK | Valores inválidos rechazados: ruta circular, % > 100, correo, sobreocupación, IATA en minúsculas y subtipo incorrecto | Superada |
| PR-03 | T-SQL | RE | Una persona sin cuenta compra un tiquete en la web | Superada |
| PR-04 | T-SQL | RN-21 | CLIENTE10: rechazado sin cuenta (50115) y aplicado al registrarse (10 %, 199,99 -> 179,99) | Superada |
| PR-05 | T-SQL | RN-19/20 | Precio histórico: la tarifa usada no se edita (50003), una nueva versión no altera ventas previas y el precio del tiquete es inmutable (50040) | Superada |
| PR-06 | T-SQL | RN-23/24 | Pago fallido sin efecto, pago parcial deja saldo y el pago final emite el tiquete | Superada |
| PR-07 | T-SQL | RN-23/25/37 | Sobrepago rechazado por el procedimiento (50135) y por el trigger (50054); efectivo en web rechazado (50055) | Superada |
| PR-08 | T-SQL | RN-12/13/14 | Asiento duplicado (50120), asiento de otra cabina (50049) y pasajero repetido en el vuelo (50121) rechazados | Superada |
| PR-09 | T-SQL | RN-11 | Sin sobreventa: la venta 13 de una cabina de 12 se rechaza; cancelar libera la plaza | Superada |
| PR-10 | T-SQL | ACID | Si falla el tiquete, la compra creada en la misma transacción también se revierte | Superada |
| PR-11 | T-SQL | RN-08/24 | Transiciones inválidas rechazadas: RESERVADO->USADO (50042), emisión sin pago (50047), PROGRAMADO->ATERRIZADO (50023) | Superada |
| PR-12 | T-SQL | RN-36 | Política de reembolso: regla de 24 h, ECO 48 h antes (0 %) y VIP 10 días antes (90 %) repartido sobre dos pagos | Superada |
| PR-13 | T-SQL | SUP-07/11 | Cancelar un vuelo cancela sus tiquetes con reembolso total y libera plazas; una reserva sin pago vence | Superada |
| PR-14 | T-SQL | RN-05/06 | Desfase horario incorrecto (50026) y llegada anterior a la salida (CHECK) rechazados; el día ISO no depende de DATEFIRST | Superada |
| PR-15 | T-SQL | RN-09/33/35 | Solape de avión (50028) y de tripulante (50034), rol incompatible (50031) y despegue sin tripulación (50029) rechazados; con tripulación completa el vuelo opera | Superada |
| PR-16 | T-SQL | RN-29 | Pagos y tiquetes no se eliminan | Superada |
| PR-17 | T-SQL | RN-03/18/32 | Programación solapada (50001), tarifa solapada (50002) y venta web con vendedor (50012) rechazadas | Superada |
| PR-18 | T-SQL | RN-04 | Generar de nuevo los vuelos de una fecha no crea duplicados | Superada |
| PR-19 | T-SQL | Seguridad | Analista sin acceso a tablas base (229) y con acceso a rpt; la aplicación no escribe directo (229); el correo se muestra enmascarado | Superada |
| PR-20 | T-SQL | Análisis | El cobro agregado por la vista coincide con la suma directa; una unión ingenua lo infla | Superada |
| PR-21 | T-SQL | RN-40 | Datos cargados: inventario conciliado, sin sobrecobros y con identidad de caja exacta | Superada |
| PR-22 | T-SQL | RN-36 | Cada reembolso cargado coincide con el que calcula ven.fn_politica_reembolso | Superada |
| PR-23 | T-SQL | Vistas/SP | rpt.usp_resumen_financiero concilia con los pagos y rpt.v_vuelo no pierde ni duplica vuelos | Superada |

| ID | Tipo | Regla | Caso probado | Resultado |
|---|---|---|---|---|
| PC-01 | Concurrencia | RN-11 | 40 sesiones simultáneas compran en una cabina VIP de 12 asientos | Superada |
| PC-02 | Concurrencia | RN-12 | 20 sesiones simultáneas eligen el asiento 12A del mismo vuelo | Superada |
| PC-03 | Concurrencia | RN-23 | 10 sesiones registran a la vez el pago total del mismo tiquete | Superada |
| PC-04 | Concurrencia | RN-12 | 30 sesiones piden a la vez la asignación automática de asiento | Superada |

*Ejecución del 2026-10-27 en `vm` (Developer Edition (64-bit), 16.0.4305.5), READ_COMMITTED_SNAPSHOT = True, conexión cifrada = TRUE. Resultado: 27 de 27 superadas. Fuente: `evidence/contenedor_linux/resultados_pruebas.json`.*

Las pruebas de concurrencia son las más ilustrativas: con 40 sesiones que intentan comprar a la vez en una cabina VIP de 12 asientos, exactamente 12 tienen éxito y 28 reciben el error de falta de disponibilidad; con 20 sesiones que piden el mismo asiento, solo una lo obtiene; con 10 pagos simultáneos del total, solo uno se confirma. La versión inicial de la asignación automática de asiento falló esta prueba —solo 6 de 30 sesiones tuvieron éxito porque todas elegían a la vez «el primer asiento libre»— y se corrigió serializando la asignación con el semáforo del vuelo y la cabina.

## Hallazgos del desarrollo

Las pruebas y las cargas detectaron 20 defectos reales que se corrigieron antes de entregar. Los más instructivos fueron: el plan de ejecución de los triggers compilado para una fila y reutilizado para cientos de miles (corregido con recompilación); un error de lógica de tres valores que permitía asignar como capitán a un empleado sin categoría (`NOT (x = 'TECNICA')` es desconocido si `x` es nulo); el efecto manada en la asignación de asientos; la transacción heredada por el *pooling* de conexiones; y una planificación sintética de tripulaciones que superaba las 100 horas de vuelo en 28 días, detectada por la propia consulta de control. El registro completo, con causa, corrección y evidencia, está en el anexo B.

## Resultados de las consultas

Se definieron 20 consultas documentadas con su pregunta de negocio, utilidad, granularidad, filtros, fórmulas, técnicas SQL, resultado ejecutado e interpretación (`docs/07_consultas/catalogo_consultas.md`). Cubren uniones internas y externas, filtros temporales, `GROUP BY` con `HAVING` y `GROUPING SETS`, subconsultas y CTE, vistas, funciones de ventana (`RANK`, `DENSE_RANK`, `LAG`, `SUM() OVER`, `PERCENTILE_CONT`), anti-joins y `OUTER APPLY`. La tabla siguiente muestra la ejecución registrada en el manifiesto.

**Tabla 10. Consultas ejecutadas desde Python con parámetros enlazados**

| ID | Consulta | Parámetros | Filas | ms |
|---|---|---|---|---|
| Q01 | Ocupación por vuelo operado | @fecha_desde = 2026-09-01, @fecha_hasta = 2026-10-31 | 1 767 | 192 |
| Q02 | Ocupación por ruta y cabina (con total por cabina) | @fecha_desde = 2026-09-01, @fecha_hasta = 2026-10-31, @min_vuelos = 20 | 46 | 170 |
| Q03 | Ventas, cobros y reembolsos por mes | @fecha_desde = 2026-07-01, | 4 | 974 |

| ID | Consulta | Parámetros | Filas | ms |
|---|---|---|---|---|
| | | @fecha_hasta = 2026-12-31 | | |
| Q04 | Saldos pendientes de reservas a la fecha de corte | — | 7 | 54 |
| Q05 | Cancelaciones con reembolso: motivo, regla y cabina | — | 8 | 1 724 |
| Q06 | Precio medio y descuento efectivo por ruta y cabina | @fecha_desde = 2026-07-01, @fecha_hasta = 2026-10-31 | 44 | 280 |
| Q07 | Ventas por canal y punto de venta | @fecha_desde = 2026-07-01, @fecha_hasta = 2026-10-31 | 18 | 299 |
| Q08 | Compradores y pasajeros registrados frente a no registrados | — | 2 | 403 |
| Q09 | Tasa de cancelación por ruta y motivo | @fecha_desde = 2026-09-01, @fecha_hasta = 2026-10-31 | 22 | 133 |
| Q10 | Utilización de la flota | @fecha_desde = 2026-09-01, @fecha_hasta = 2026-10-31 | 17 | 25 |
| Q11 | Distribución de la demanda por día de la semana y franja horaria | @fecha_desde = 2026-09-01, @fecha_hasta = 2026-10-31 | 28 | 140 |
| Q12 | Anticipación de la compra y precio pagado | @fecha_desde = 2026-09-01, @fecha_hasta = 2026-10-31 | 5 | 133 |
| Q13 | Clientes registrados de mayor valor | @top = 10 | 10 | 550 |
| Q14 | Registros sin actividad (anti-joins) | — | 5 | 83 |
| Q15 | Puntualidad y cancelaciones operativas por ruta | @fecha_desde = 2026-09-01, @fecha_hasta = 2026-10-31 | 22 | 49 |
| Q16 | Disponibilidad de los próximos vuelos | @dias = 3, @origen = None | 170 | 52 |
| Q17 | Evolución mensual de las ventas (variación y acumulado) | — | 8 | 115 |
| Q18 | Carga de vuelo de la tripulación en 672 horas consecutivas | @fecha_desde = 2026-10-01, @fecha_hasta = 2026-10-28 | 4 | 33 |
| Q19 | Controles de integridad y conciliación | — | 9 | 960 |
| Q20 | Uso y costo de los descuentos | — | 6 | 190 |

*Ejecución del 2026-10-27 en `vm` (Developer Edition (64-bit), 16.0.4305.5). Fuente: `reports/resultados/manifiesto_consultas.json`, que guarda también la huella SHA-256 de cada SQL.*

**Ventas, cobros y reembolsos.** La figura 7 y la tabla siguiente separan tres magnitudes que suelen confundirse: la venta (precio final de los tiquetes creados en el mes), el cobro (pagos confirmados por su fecha de confirmación) y el reembolso (dinero devuelto por su fecha). Cada una se agrega por su propia fecha en una expresión separada, porque unir pagos y reembolsos antes de agregar multiplicaría los importes: una prueba específica (PR-20) muestra que una unión ingenua infla el cobro de 72,27 a 87,78 millones de USD.

**Tabla 11. Ventas, cobros y reembolsos por mes (Q03)**

| Mes | Tiquetes vendidos | Ventas brutas | Ventas vigentes | Cobros confirmados | Reembolsos | Cobro neto |
|---|---|---|---|---|---|---|
| jul 2026 | 48 392 | 9 965 346,08 | 9 197 336,45 | 9 774 572,77 | 219 755,72 | 9 554 817,05 |
| ago 2026 | 70 363 | 15 591 245,82 | 14 441 036,70 | 15 285 358,39 | 482 237,63 | 14 803 120,76 |
| sep 2026 | 103 529 | 23 736 794,26 | 22 104 440,77 | 23 374 990,21 | 1 001 647,25 | 22 373 342,96 |
| oct 2026 | 101 434 | 24 224 876,63 | 22 647 526,23 | 23 833 321,92 | 1 327 161,44 | 22 506 160,48 |

*Importes en USD. Fuente: `reports/resultados/Q03_ventas_cobros_reembolsos_mes.csv` (consulta `sql/consultas/Q03_ventas_cobros_reembolsos_mes.sql`). Datos sintéticos.*

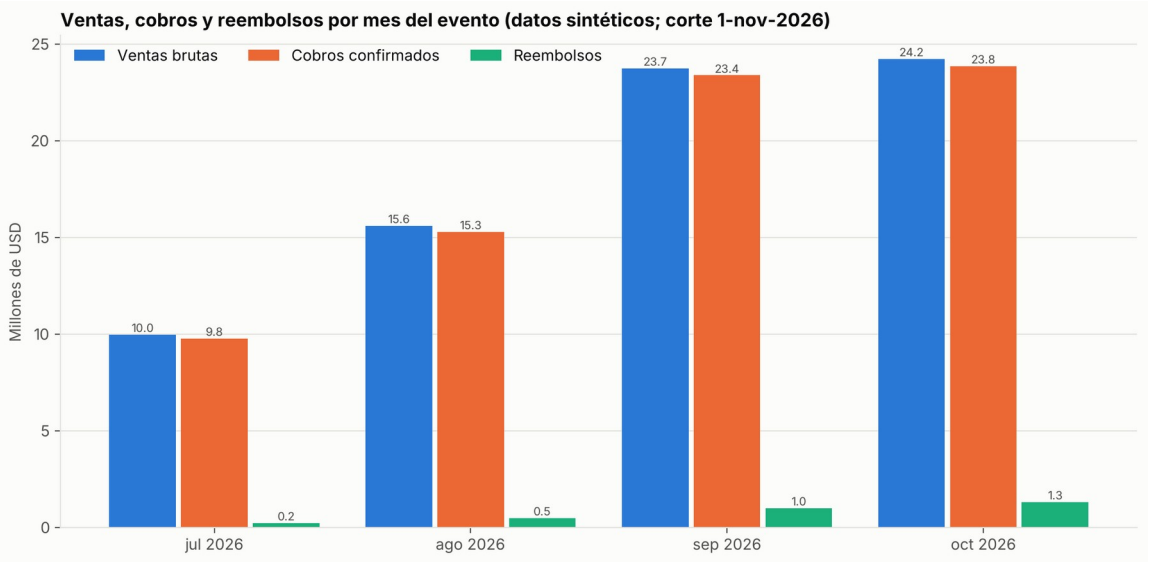



*Figura 7. Ventas, cobros y reembolsos por mes (Q03). Fuente: `reports/figuras/F02_flujo_mensual.png`.*

Los cobros del período (72,27 millones de USD) superan a las ventas vigentes (68,39 millones) sin que haya error: incluyen dinero que luego se devolvió y penalidades retenidas por cancelaciones. La consulta de controles (Q19) verifica la identidad completa —cobros menos reembolsos igual a ventas emitidas más abonos de reservas vigentes más penalidades retenidas— con una diferencia de 0,00 USD. No se calcula rentabilidad, porque el modelo no registra costos.

**Saldos pendientes.** A la fecha de corte quedan 824 reservas por pagar, con 122 943,56 USD de saldo; 118 vencen en menos de 24 horas.

**Tabla 12. Reservas con saldo pendiente a la fecha de corte (Q04)**

| Situación | Plazo hasta el vencimiento | Tiquetes | Valor reservado | Abonado | Saldo pendiente |
|---|---|---|---|---|---|
| 1. Sin pagos | a. vence en menos de 24 h | 40 | 9 955,55 | 0,00 | 9 955,55 |
| 1. Sin pagos | b. vence en 24-48 h | 44 | 10 013,56 | 0,00 | 10 013,56 |
| 1. Sin pagos | c. vence en 48 h o más | 64 | 15 357,21 | 0,00 | 15 357,21 |
| 2. Pago en espera de confirmación | c. vence en 48 h o más | 11 | 2 466,14 | 0,00 | 2 466,14 |
| 3. Pago parcial | a. vence en menos de 24 h | 78 | 17 536,07 | 8 497,72 | 9 038,35 |
| 3. Pago parcial | b. vence en 24-48 h | 225 | 56 266,65 | 27 204,83 | 29 061,82 |
| 3. Pago parcial | c. vence en 48 h o más | 362 | 87 919,23 | 40 868,30 | 47 050,93 |

*Importes en USD. Fuente: `reports/resultados/Q04_saldos_pendientes.csv` (consulta `sql/consultas/Q04_saldos_pendientes.sql`). Datos sintéticos.*

**Cancelaciones y reembolsos.** Se cancelaron 22 728 tiquetes; de 4,00 millones de USD cobrados en ellos se devolvieron 3,03 millones. La regla de 24 horas y las cancelaciones de vuelo reembolsan el 100 %; las cancelaciones voluntarias fuera de esa regla siguen la política por cabina, con un porcentaje medio aplicado de 56,8 % en económica y 79,9 % en VIP. La regla y el porcentaje se

recalculan con la misma función que usa el procedimiento de cancelación, y la prueba PR-22 confirma que coinciden en los 22 728 casos. La tasa de reembolso se define como nula cuando no hubo cobro, para distinguir «no había nada que devolver» de «se devolvió el 0 %».

**Tabla 13. Cancelaciones y reembolsos por motivo, regla y cabina (Q05)**

| Motivo | Regla aplicada | Cabina | Tiquetes | % política medio | Cobrado | Reembolsado | Retenido | Tasa de reembolso |
|---|---|---|---|---|---|---|---|---|
| VENCIMIENTO | REEMBOLSO_TOTAL | ECO | 4 467 | 100,0 % | 48 400,06 | 48 400,06 | 0,00 | 100,0 % |
| VENCIMIENTO | REEMBOLSO_TOTAL | VIP | 365 | 100,0 % | 12 389,12 | 12 389,12 | 0,00 | 100,0 % |
| VOLUNTARIA | POLITICA_CABINA | ECO | 10 101 | 56,8 % | 1 989 653,45 | 1 122 239,25 | 867 414,20 | 56,4 % |
| VOLUNTARIA | POLITICA_CABINA | VIP | 909 | 79,9 % | 502 673,14 | 400 042,68 | 102 630,46 | 79,6 % |
| VOLUNTARIA | REGLA_24H | ECO | 4 434 | 100,0 % | 829 879,38 | 829 879,38 | 0,00 | 100,0 % |
| VOLUNTARIA | REGLA_24H | VIP | 387 | 100,0 % | 196 456,56 | 196 456,56 | 0,00 | 100,0 % |
| VUELO_CANCELADO | REEMBOLSO_TOTAL | ECO | 1 878 | 100,0 % | 329 667,28 | 329 667,28 | 0,00 | 100,0 % |
| VUELO_CANCELADO | REEMBOLSO_TOTAL | VIP | 187 | 100,0 % | 91 727,71 | 91 727,71 | 0,00 | 100,0 % |

*Importes en USD. Tasa de reembolso = reembolsado / cobrado (nula si no hubo cobro). Fuente:* `reports/resultados/Q05_reembolsos.csv` *(consulta* `sql/consultas/Q05_reembolsos.sql`*). Datos sintéticos.*

**Ocupación.** El factor de ocupación ponderado del período es 68,9 % en económica y 53,0 % en VIP (figura 8). El factor ponderado (asientos usados entre asientos ofrecidos) difiere ligeramente del promedio simple de los factores por vuelo porque los aviones tienen capacidades distintas; para la red corresponde el ponderado. La distribución por vuelo (figura 9) tiene mediana de 67,0 %, con 77 vuelos por encima del 85 % y 92 por debajo del 50 %.

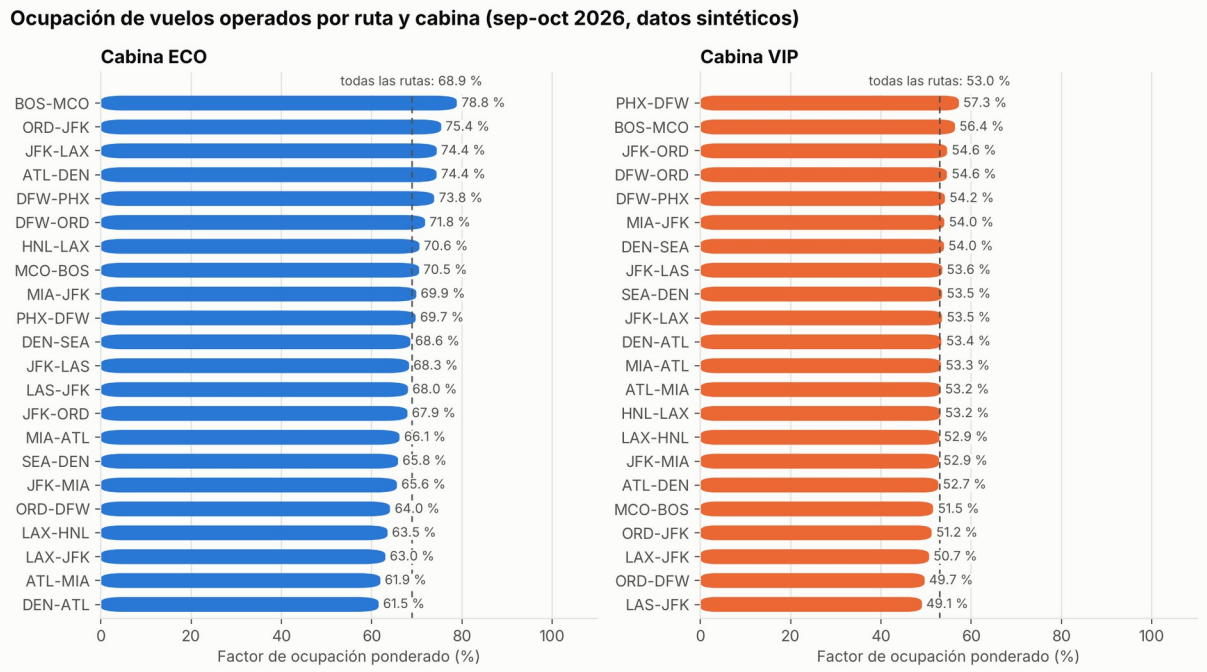



*Figura 8. Ocupación por ruta y cabina (Q02). Fuente:* `reports/figuras/F01_ocupacion_ruta_cabina.png`.

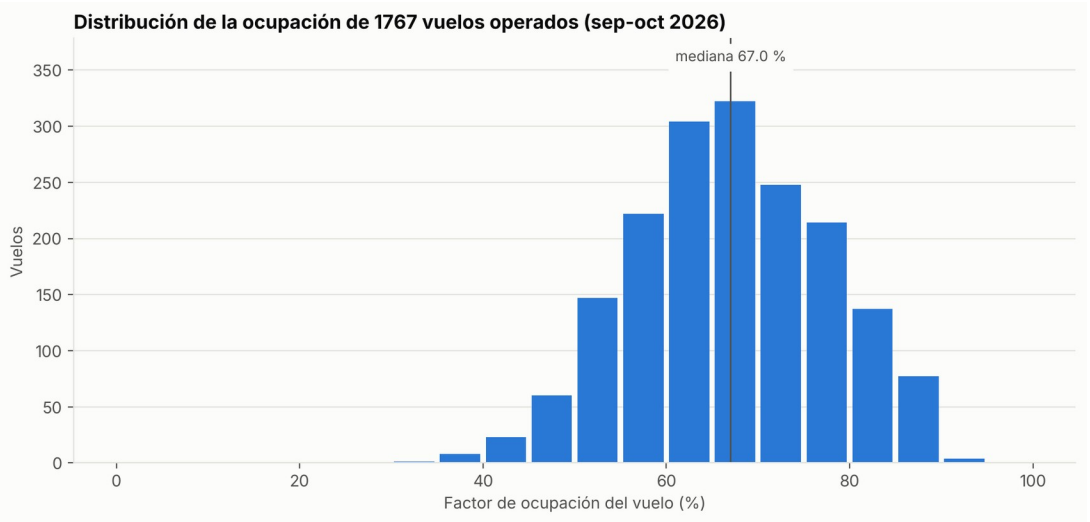



*Figura 9. Distribución de la ocupación por vuelo (Q01). Fuente:* `reports/figuras/F07_distribucion_ocupacion.png`.

**Demanda en el tiempo.** La ocupación es mayor los viernes y domingos y menor martes y miércoles (figura 10), patrón que reproduce los factores por día de la semana del generador y valida el cálculo del día ISO, independiente de la configuración regional de la sesión. Respecto de la anticipación (figura 11), el precio medio en económica baja de 206,82 USD para compras con menos de siete días a 179,94 USD para compras con 60 días o más, mientras la penetración de descuentos sube de 13,6 % a 47,1 %. La asociación es descriptiva: se explica por cómo se construyeron las tarifas y el descuento por anticipación, y no demuestra que comprar antes cause un precio menor en un mercado real.

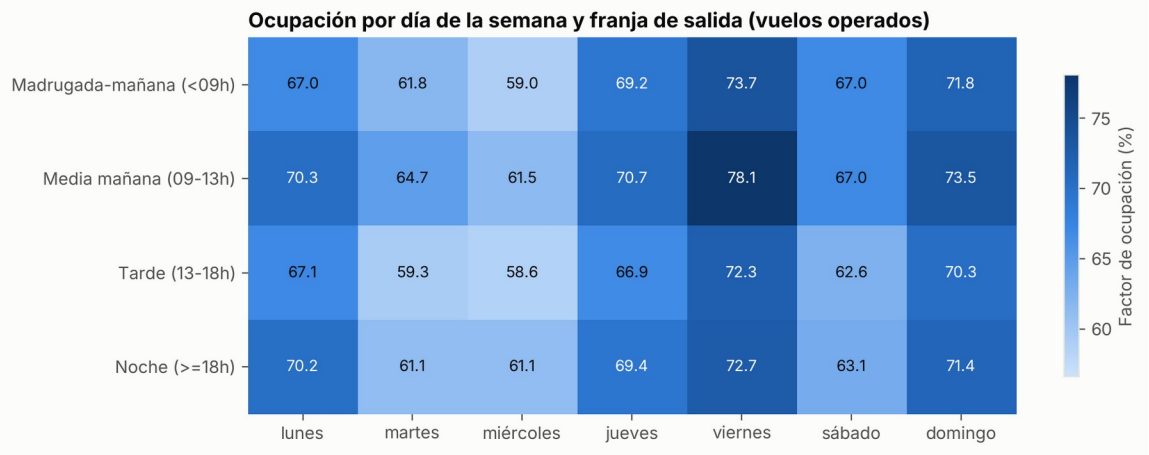



*Figura 10. Ocupación por día de la semana y franja de salida (Q11). Fuente:* `reports/figuras/F06_demanda_dia_franja.png`.

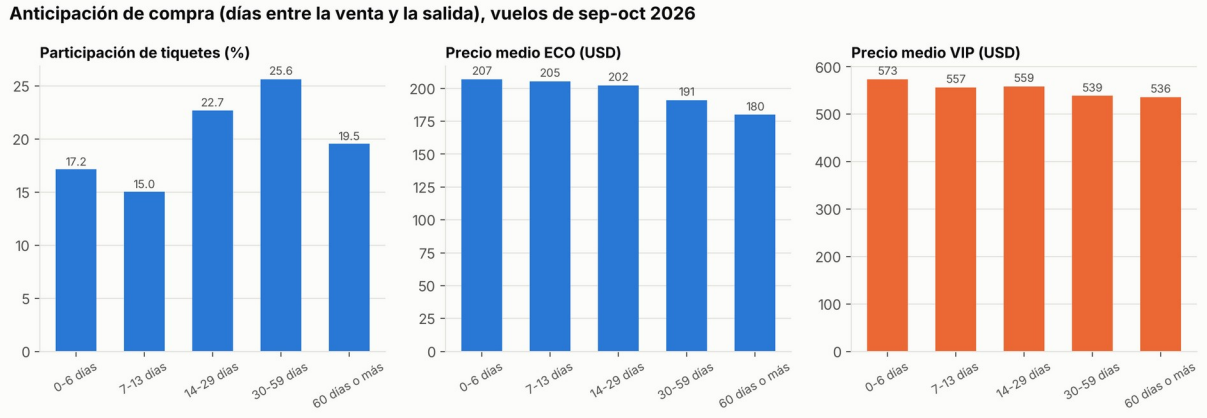



*Figura 11. Anticipación de compra y precio (Q12). Fuente:* `reports/figuras/F04_anticipacion_compra.png`.

**Tabla 14. Anticipación de la compra y precio medio pagado (Q12)**

| Anticipación | Tiquetes | Participación | Precio medio ECO (USD) | Precio medio VIP (USD) | Con descuento |
|---|---|---|---|---|---|
| 1. 0-6 días | 34 958 | 17,2 % | 206,82 | 573,34 | 13,6 % |
| 2. 7-13 días | 30 675 | 15,0 % | 205,31 | 556,84 | 13,8 % |
| 3. 14-29 días | 46 210 | 22,7 % | 202,26 | 558,59 | 13,2 % |
| 4. 30-59 días | 52 182 | 25,6 % | 191,05 | 538,76 | 25,9 % |
| 5. 60 días o más | 39 821 | 19,5 % | 179,94 | 536,27 | 47,1 % |

*Vuelos operados en septiembre y octubre de 2026. Fuente:* `reports/resultados/Q12_anticipacion_compra.csv` *(consulta* `sql/consultas/Q12_anticipacion_compra.sql`*). Datos sintéticos.*

**Canales y clientes.** El canal web concentra el 70,0 % de las ventas vigentes (figura 12). El 41,9 % de las compras las hizo un comprador registrado, valor que recupera el 42 % del generador y valida la lógica de «registrado en el momento de la compra». Las compras sin registro no se pueden atribuir de forma fiable a una misma persona, una limitación inherente a ese tipo de compra. Precio medio, tasa de cancelación y uso del canal web son casi iguales entre registrados y no registrados porque el

generador no los diferencia; con datos reales, esta consulta sería el punto de partida para compararlos.

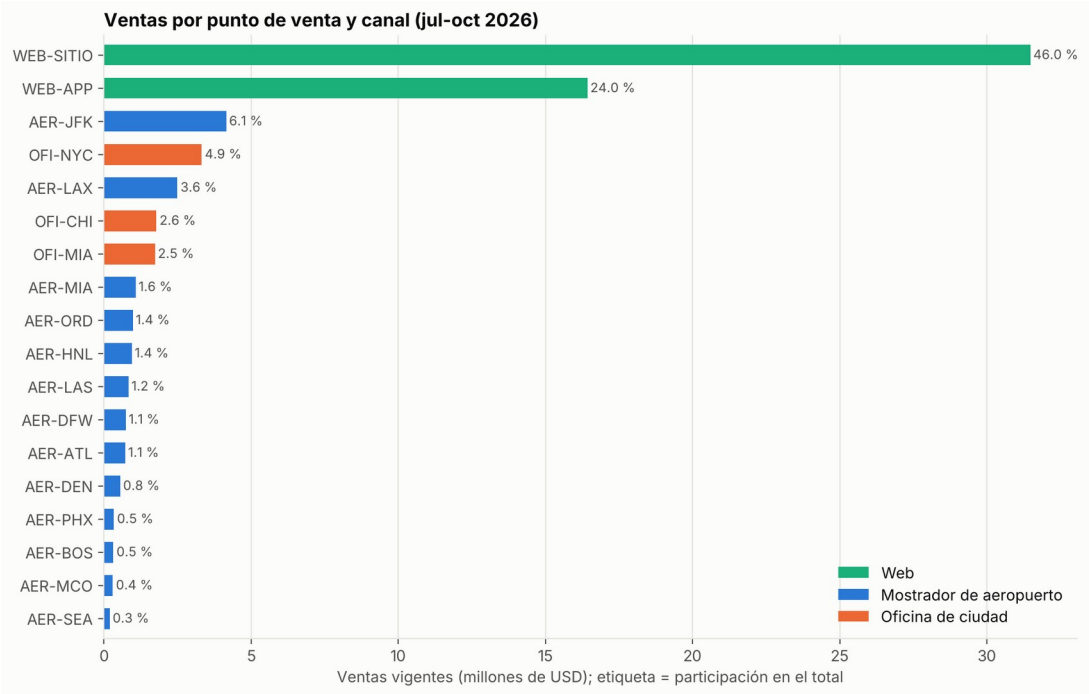



*Figura 12. Ventas por punto de venta y canal (Q07). Fuente: `reports/figuras/F03_ventas_canal.png`.*

**Tabla 15. Compradores registrados frente a no registrados (Q08)**

| Comprador | Compras | Personas distintas | Tiquetes | Tiquetes por compra | Ventas brutas (USD) | Precio medio (USD) | Tasa de cancelación | Canal web |
|---|---|---|---|---|---|---|---|---|
| Registrado | 70 828 | 22 701 | 135 656 | 1,92 | 30 483 960,07 | 224,72 | 7,0 % | 69,9 % |
| No registrado | 98 100 | 98 100 | 188 062 | 1,92 | 43 034 302,72 | 228,83 | 7,0 % | 70,1 % |

*Fuente: `reports/resultados/Q08_registrados_vs_no_registrados.csv` (consulta `sql/consultas/Q08_registrados_vs_no_registrados.sql`). Datos sintéticos.*

**Precios y descuentos.** La tasa de descuento efectiva, ponderada por importe, es de 3,4 % a 3,8 % en económica y de 0,9 % a 1,5 % en VIP, muy por debajo de los porcentajes nominales (10 % a 20 %) porque se reparte entre todos los tiquetes. El descuento para clientes VIP registrados (`VIPPLUS5`) nunca se aplica: con la política de un solo descuento, el de mayor porcentaje, todo comprador que califica para él califica también para `CLIENTE10`. No es un error de datos, sino una consecuencia de la regla que convendría revisar al diseñar promociones.

**Tabla 16. Uso y costo de los descuentos (Q20)**

| Código | % | Cabina | Solo registrados | Tiquetes | Costo (USD) | Compradores registrados |
|---|---|---|---|---|---|---|
| ANTICIPA15 | 15,0 | ECO | no | 25 427 | 782 256,45 | 42,3 % |
| LANZAMIENTO20 | 20,0 | ECO | no | 22 713 | 908 578,40 | 40,9 % |
| CLIENTE10 | 10,0 | — | sí | 9 296 | 246 608,70 | 100,0 % |
| OTONO12 | 12,0 | — | no | 8 851 | 276 077,40 | 42,7 % |
| INVIERNO25 | 25,0 | — | no | 0 | 0,00 | — |

| Código | % | Cabina | Solo registrados | Tiquetes | Costo (USD) | Compradores registrados |
|---|---|---|---|---|---|---|
| VIPPLUS5 | 5,0 | VIP | sí | 0 | 0,00 | — |

*Tiquetes no cancelados. Cabina vacía = todas. Fuente:* `reports/resultados/Q20_uso_descuentos.csv` *(consulta* `sql/consultas/Q20_uso_descuentos.sql`*). Datos sintéticos.*

**Operación.** De 1 767 vuelos operados, el 78,4 % llegó con menos de 15 minutos de demora (figura 13), criterio que usa la estadística oficial de puntualidad de Estados Unidos (Bureau of Transportation Statistics, s. f.); se cancelaron 20 vuelos (1,1 %). Los A321neo de las líneas transcontinentales promedian de 10,0 a 11,2 horas de bloque diarias y los aviones de reserva solo vuelan los días de mantenimiento del titular (figura 14).

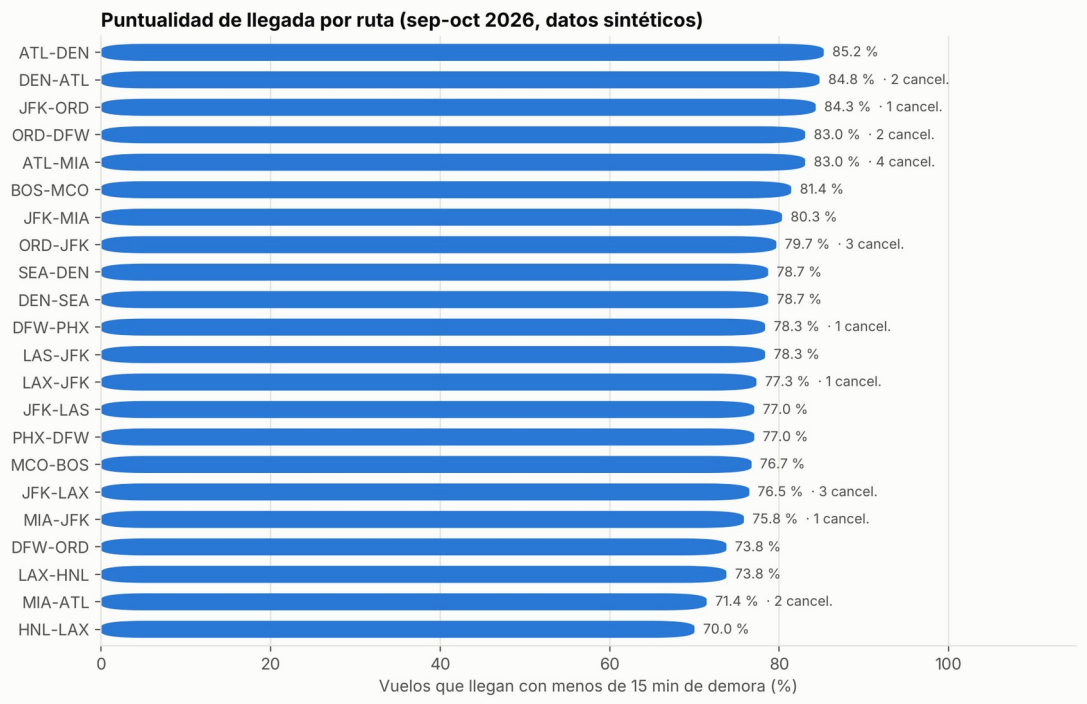



*Figura 13. Puntualidad de llegada por ruta (Q15). Fuente:* `reports/figuras/F05_puntualidad_ruta.png`.

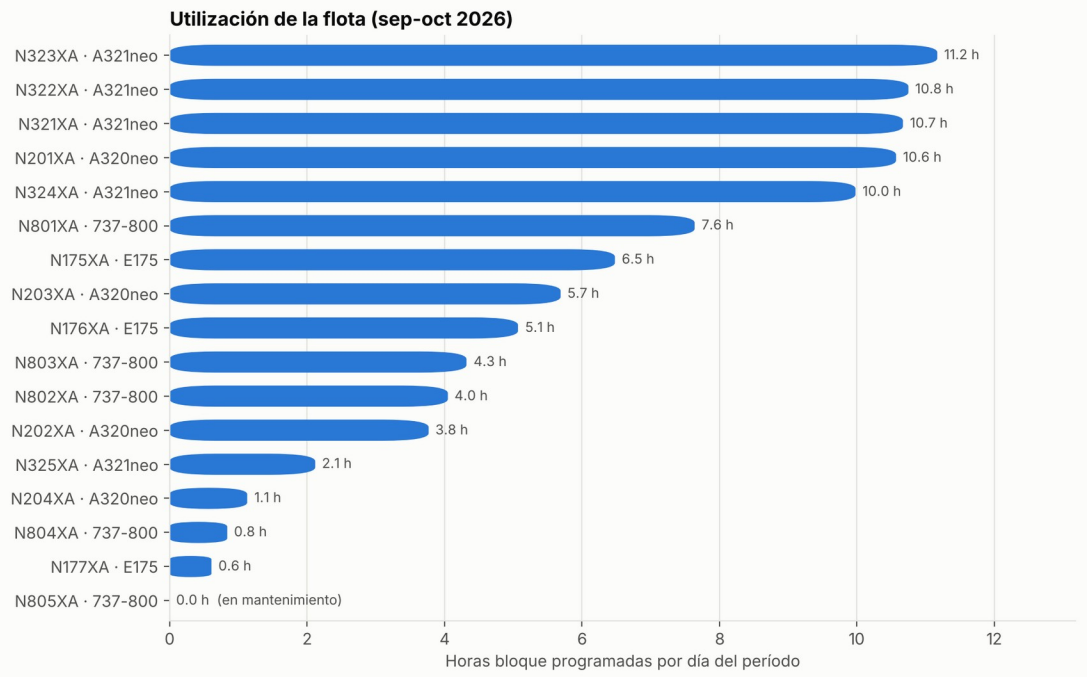



*Figura 14. Utilización de la flota (Q10). Fuente: `reports/figuras/F08_utilizacion_flota.png`.*

**Tripulación.** En la ventana de 672 horas consecutivas del 1 al 28 de octubre, ningún piloto supera las 100 horas de vuelo, la referencia de la normativa estadounidense (Cumulative Limitations, 2026); el máximo es 94,5 horas. La primera versión de los datos sintéticos sí la superaba (188 horas de máximo con dos equipos por línea y 105,8 con cuatro), y fue esta consulta la que lo detectó. La base no impone el límite; la consulta es un control a posteriori sobre una ventana.

**Tabla 17. Horas de vuelo de la tripulación en 672 horas consecutivas (Q18)**

| Rol | Tripulantes | Asignaciones | Horas mín. | Horas medias | Horas máx. | Pilotos > 100 h | Vuelos sin 2 pilotos |
|---|---|---|---|---|---|---|---|
| AUXILIAR_VUELO | 140 | 2 115 | 17,3 | 52,1 | 94,5 | 0 | 0 |
| CAPITAN | 52 | 815 | 17,3 | 51,1 | 94,5 | 0 | 0 |
| JEFE_CABINA | 52 | 815 | 17,3 | 51,1 | 94,5 | 0 | 0 |
| PRIMER_OFICIAL | 52 | 815 | 17,3 | 51,1 | 94,5 | 0 | 0 |

*Ventana del 1 al 28 de octubre de 2026; horas de bloque programadas. Fuente: `reports/resultados/Q18_tripulacion_horas.csv` (consulta `sql/consultas/Q18_tripulacion_horas.sql`). Datos sintéticos.*

**Integridad.** Los nueve controles de Q19 devuelven cero diferencias:

**Tabla 18. Controles de integridad y conciliación (Q19)**

| Control | Filas con diferencia | Diferencia (USD) |
|---|---|---|
| Asientos duplicados en tiquetes activos | 0 | — |
| Cobros confirmados mayores que el precio final | 0 | — |
| Identidad de conciliación de caja (USD) | 0 | 0,00 |
| Inventario distinto de tiquetes que ocupan plaza | 0 | — |
| Plazas ocupadas por encima de la capacidad | 0 | — |
| Reembolsos mayores que lo cobrado | 0 | — |

| Control | Filas con diferencia | Diferencia (USD) |
|---|---|---|
| Tiquetes con precio base distinto de su tarifa | 0 | — |
| Tiquetes emitidos, usados o no presentados sin pago total | 0 | — |
| Vuelos operados con reservas sin pagar | 0 | — |

*Fuente:* `reports/resultados/Q19_controles_conciliacion.csv` *(consulta* `sql/consultas/Q19_controles_conciliacion.sql`*). Datos sintéticos.*

## Discusión

Los resultados respaldan la decisión central del diseño: poner las reglas en el motor. La carga masiva, las pruebas de reglas y las de concurrencia ejercen las mismas restricciones y triggers por caminos distintos (sentencias de conjunto, procedimientos y sesiones simultáneas), y en ninguno se pudo introducir un dato inválido. El costo de esa decisión fue de rendimiento y se hizo visible en la carga: los triggers escritos para una fila no escalaban a cientos de miles hasta reescribirlos en forma de conjuntos, con recompilación y sin el «objetivo de filas» del optimizador.

El análisis muestra también por qué importan las definiciones: ventas, cobros y reembolsos difieren legítimamente; el factor ponderado y el promedio simple de ocupación responden a preguntas distintas; un nulo y un cero significan cosas distintas en la tasa de reembolso. Las vistas de reporte fijan esas definiciones una sola vez.

Por último, la mayoría de los patrones analíticos reproduce supuestos del generador. Eso es útil para validar las consultas —si la consulta no recuperara el 42 % de compradores registrados, estaría mal—, pero impide extraer conclusiones de negocio. Con datos reales, las mismas consultas se convertirían en herramientas de decisión.

# Seguridad, limitaciones y trabajo futuro

## Seguridad

Se definieron cuatro roles con privilegio mínimo: el analista solo lee el esquema de reportes, ventas y operaciones ejecutan sus procedimientos, y la cuenta de la aplicación ejecuta procedimientos pero tiene denegada la escritura directa en las tablas de ventas y operaciones. Esto funciona por la cadena de propiedad: como todos los esquemas pertenecen a `dbo`, quien puede ejecutar un procedimiento puede modificar las tablas a través de él sin tener permiso sobre ellas. Seis columnas personales (correos, teléfono, documento, fecha de nacimiento y salario) se enmascaran para usuarios sin privilegio. De las tarjetas solo se guardan los últimos cuatro dígitos y la marca. Las conexiones van siempre cifradas, el repositorio no contiene secretos y los pagos, reembolsos y tiquetes no se eliminan: los errores se corrigen con nuevas filas y cada cambio de estado queda auditado.

## Limitaciones

- **Nube pendiente.** El requisito de base de datos en la nube no está cumplido mientras no se despliegue en Azure SQL Database con evidencia propia.
- **Entorno del equipo pendiente.** La validación se hizo en un contenedor Linux con SQL Server 2022; falta repetirla en la laptop Windows, donde también deben comprobarse la autenticación de Windows y SSMS.
- **Datos sintéticos.** Las cifras no describen un mercado real y varios patrones reproducen supuestos del generador.
- **Versiones del motor.** La compatibilidad con SQL Server 2016 se comprobó con el nivel de compatibilidad 130 en un motor 2022, no en un motor 2016 real.
- **Enmascaramiento no es cifrado.** Un usuario con consultas ad hoc puede inferir valores enmascarados.
- **Normativa.** No se modelan impuestos, límites de servicio de tripulación ni otras obligaciones regulatorias, salvo como controles por consulta.
- **Rendimiento.** Los tiempos medidos son del contenedor y no constituyen una prueba de carga.

## Trabajo futuro

Desplegar en Azure y comparar tiempos y planes con el entorno local; cifrar columnas sensibles con *Always Encrypted*; modelar conexiones y tarifas combinadas; incorporar costos para analizar rentabilidad por ruta; sustituir los datos sintéticos por datos abiertos de operación (por ejemplo, los de puntualidad del Departamento de Transporte) para análisis reales; automatizar las pruebas en un

servicio de integración continua con un contenedor de SQL Server; e imponer en la base los límites de servicio de tripulación con ventanas móviles.

# Conclusiones

1. **OE1 · Requisitos.** Los requisitos del enunciado y de las guías se tradujeron en 42 reglas verificables y 13 supuestos explícitos, y las doce discrepancias entre documentos quedaron registradas sin asumir cuál prevalece. Las ambigüedades principales —pasajero frente a comprador, horario frente a vuelo, precio histórico, combinación de descuentos y pagos parciales— tienen una decisión documentada y una consecuencia concreta en el modelo.
2. **OE2 · Modelo.** El modelo conceptual, lógico y físico es coherente por construcción: el diccionario y los diagramas físicos se generan del catálogo del motor. El esquema está en tercera forma normal y las seis redundancias aceptadas tienen un mecanismo que impide sus anomalías, como lo demuestran los controles de conciliación con cero diferencias.
3. **OE3 · Integridad.** La base rechaza los datos inválidos en todos los casos probados: 23 de 23 pruebas de reglas y 4 de 4 de concurrencia superadas, sin sobreventa, asientos duplicados ni cobros dobles con sesiones simultáneas reales. La preferencia por restricciones declarativas y el uso de bloqueos sobre el recurso padre permitieron, además, cargar casi 1,2 millones de filas con todas las reglas activas en unos 90 segundos.
4. **OE4 · Datos y consultas.** Los datos sintéticos son reproducibles con una semilla y verificables por huella. Las 20 consultas documentadas se ejecutan desde Python con parámetros enlazados, exportan CSV y alimentan las figuras del informe; recuperan los parámetros conocidos del generador, lo que valida su lógica, y distinguen ventas, cobros y reembolsos sin multiplicar importes.
5. **OE5 · Validación y nube.** La validación local es amplia y está documentada con evidencias, incluidos 20 defectos reales corregidos. El despliegue en Azure SQL Database está preparado pero **no ejecutado**; hasta que el equipo lo realice con autorización, el requisito de nube del encargo debe considerarse pendiente.

## Recomendaciones

1. Ejecutar la verificación del entorno y la reproducción completa en la laptop Windows (`scripts/windows/verificar_entorno.ps1` y `python -m aerolinea reproducir`) y guardar las evidencias en `evidence/laptop_windows/`.
2. Desplegar en Azure SQL Database con la oferta gratuita y pausa automática, siguiendo `docs/05_azure/guia_azure.md`, y completar la lista de evidencias antes de la entrega.
3. Confirmar con el docente el número mínimo de páginas y la semana de entrega del avance, y completar los campos marcados como [POR COMPLETAR].
4. Publicar el repositorio en GitHub desde las cuentas del equipo y registrar los aportes reales de cada integrante.
5. Preparar la defensa con la guía de estudio común: como la exposición es aleatoria, cada integrante debe poder explicar cualquier sección.
6. Si el proyecto continúa, priorizar el cifrado de columnas sensibles y el uso de datos reales para que las consultas pasen de validar el modelo a apoyar decisiones.

# Referencias

Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A critique of ANSI SQL isolation levels. En *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data* (pp. 1–10). Association for Computing Machinery. https://doi.org/10.1145/223784.223785

Bureau of Transportation Statistics. (s. f.). *Flight operations arriving on time: Largest U.S. air carriers*. U.S. Department of Transportation. Recuperado el 27 de octubre de 2026, de https://www.bts.gov/content/flight-operations-arriving-time-largest-us-air-carriersa

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, *13*(6), 377–387. https://doi.org/10.1145/362384.362685

Cumulative limitations, 14 C.F.R. § 117.23 (2026). https://www.ecfr.gov/current/title-14/chapter-I/subchapter-G/part-117/section-117.23

Customer service plan, 14 C.F.R. § 259.5 (2026). https://www.ecfr.gov/current/title-14/chapter-II/subchapter-A/part-259/section-259.5

Elmasri, R., & Navathe, S. B. (2011). *Fundamentals of database systems* (6th ed.). Addison-Wesley.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Microsoft. (s. f.-a). *AT TIME ZONE (Transact-SQL)*. Microsoft Learn. Recuperado el 27 de octubre de 2026, de https://learn.microsoft.com/en-us/sql/t-sql/queries/at-time-zone-transact-sql

Microsoft. (s. f.-b). *Create filtered indexes*. Microsoft Learn. Recuperado el 27 de octubre de 2026, de https://learn.microsoft.com/en-us/sql/relational-databases/indexes/create-filtered-indexes

Microsoft. (s. f.-c). *Deploy Azure SQL Database for free*. Microsoft Learn. Recuperado el 27 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/azure-sql/database/free-offer

Microsoft. (s. f.-d). *Dynamic data masking*. Microsoft Learn. Recuperado el 27 de octubre de 2026, de https://learn.microsoft.com/en-us/sql/relational-databases/security/dynamic-data-masking

Microsoft. (s. f.-e). *Transaction locking and row versioning guide*. Microsoft Learn. Recuperado el 27 de octubre de 2026, de https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide

Microsoft. (2023). *¿Qué son las bases de datos?* Microsoft Azure. https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-are-databases

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill Education.

Universidad Continental. (2026). *Producto académico colaborativo: Gestión de datos y bases de datos con SQL (PSMA00611)* [Enunciado del aula virtual]. Escuela de Posgrado.

Yaseen, A. (2019). *Restricciones en SQL Server* [Lectura del curso]. SQL Shack. https://www.sqlshack.com/es/restricciones-en-sql-server-sql-not-null-unique-y-sql-primary-key/

# Anexos

## Anexo A. Matriz de trazabilidad

Requisitos del encargo, documento y página de origen, archivos que los implementan, evidencia y estado. Tipo: OBL = obligatorio según las fuentes; MEJ = mejora propuesta por el equipo. La versión completa está en `docs/00_fuentes/matriz_requisitos.md`.

**Tabla 19. Matriz de trazabilidad de requisitos**

| ID | Requisito | Documento y página | Tipo | Archivo(s) | Estado |
|---|---|---|---|---|---|
| R01 | Base de datos **en la nube** | F1; F3 p. 21; F4 p. 21 | OBL | `docs/05_azure/guia_azure.md`, `scripts/azure/crear_recursos_azure.ps1`, `sql/azure/verificacion_azure.sql`, `.env.azure.example` | **Pendiente**: preparado; no se creó ningún recurso (requiere suscripción y autorización) |
| R02 | Definición de los requisitos de la empresa | F1; F3 p. 22; F4 p. 21 | OBL | `docs/01_requisitos/analisis_requisitos.md` | Cumplido |
| R03 | Diseño del modelo de datos | F1; F3 p. 22; F4 p. 21 | OBL | `docs/02_modelo/` (conceptual, lógico, físico, normalización) | Cumplido |
| R04 | Creación de tablas | F1; F3 p. 22; F4 p. 21 | OBL | `sql/02-sql/04` | Cumplido (contenedor) |
| R05 | Inserción de datos | F1; F3 p. 22; F4 p. 21 | OBL | `sql/09_datos_referencia.sql`, `src/aerolinea/generador.py`, `sql/10_carga_staging.sql` | Cumplido (contenedor) |
| R06 | Definición de consultas | F1; F3 p. 22; F4 p. 21 | OBL | `sql/consultas/Q01-Q20` | Cumplido (contenedor) |
| R07 | Implementación de las consultas en un lenguaje de programación | F1; F3 p. 22; F4 p. 21 | OBL | `src/aerolinea/consultas.py`, `operaciones.py` | Cumplido (contenedor) |
| R08 | Fundamentos y DML | F1 (logro); F3 pp. 21-22; F4 p. 21 | OBL | `sql/07_procedimientos.sql`, `sql/11_dml_seguro.sql` | Cumplido (contenedor) |
| R09 | Restricciones | F1 (logro); F3 pp. 21-22; F4 p. 21 | OBL | `sql/03_claves_y_restricciones.sql`, `sql/05_funciones_y_triggers.sql` | Cumplido (contenedor) |
| R10 | Agrupaciones y agregaciones | F1 (logro); F3 pp. 21-22; F4 p. 21 | OBL | Q02, Q03, Q06, Q07, Q09, Q11, Q12, Q17, Q20 | Cumplido (contenedor) |
| R11 | Vistas | F1 (logro); F3 pp. 21-22; F4 p. 21 | OBL | `sql/06_vistas.sql` | Cumplido (contenedor) |
| R12 | Procedimientos | F1 (logro); F3 | OBL | `sql/` | Cumplido (contenedor) |

| ID | Requisito | Documento y página | Tipo | Archivo(s) | Estado |
|---|---|---|---|---|---|
| | almacenados | pp. 21-22; F4 p. 21 | | `07_procedimientos.sql` | |
| R13 | Clientes | F1; F3 p. 22; F4 p. 21 | OBL | `ven.persona`, `ven.cliente` | Cumplido |
| R14 | Empleados | F1; F3 p. 22; F4 p. 21 | OBL | `rh.empleado`, `rh.cargo`, `rh.departamento` | Cumplido |
| R15 | Rutas con origen, destino, día de la semana y hora prevista | F1; F3 p. 22; F4 p. 21 | OBL | `ops.ruta`, `ops.programacion`, `ops.programacion_dia` | Cumplido |
| R16 | Vuelos generados a partir de una ruta programada | F1; F3 p. 22; F4 p. 21 | OBL | `ops.usp_generar_vuelos` | Cumplido (contenedor) |
| R17 | Localizaciones físicas y web de compra | F1; F3 p. 22; F4 p. 21 | OBL | `ref.localizacion` (AER, OFI, WEB), `ref.aeropuerto` | Cumplido |
| R18 | Tiquetes comprados por clientes registrados o no | F1; F3 p. 22; F4 p. 21 | OBL | `ven.compra`, `ven.tiquete` | Cumplido |
| R19 | Aviones asignados a los viajes | F1; F3 p. 22; F4 p. 21 | OBL | `ops.avion`, `ops.usp_asignar_avion` | Cumplido |
| R20 | Cabina VIP o económica; precio por ruta y cabina | F1; F3 p. 22; F4 p. 21 | OBL | `ref.cabina`, `ven.tarifa` (versionada) | Cumplido |
| R21 | Uno o más pagos por tiquete | F1; F3 p. 22; F4 p. 21 | OBL | `ven.pago`, `ven.usp_registrar_pago` | Cumplido |
| R22 | Descuentos porcentuales sobre el precio original | F1; F3 p. 22; F4 p. 21 | OBL | `ven.descuento`, columnas calculadas en `ven.tiquete` | Cumplido |
| R23 | Ser exhaustivo con los atributos | F1; F3 p. 22; F4 p. 21 | OBL | `docs/02_modelo/diccionario_datos.md` (269 columnas con definición) | Cumplido |
| R24 | Scripts en GitHub (registrarse, crear equipo y proyecto, subir scripts) | F3 p. 22; F4 p. 22 (no figura en F1) | OBL (guías) | Estructura del repositorio, `README.md`, `.gitignore`, `CONTRIBUTING.md` | **Pendiente**: el repositorio está listo localmente; crear la organización o el repositorio y subirlo requiere cuentas del equipo y su autorización |
| R25 | Presentación de 10 a 15 diapositivas, 20 minutos | F3 p. 22; F4 p. 22 | OBL (guías) | `slides/presentacion_PSMA00611.pptx`, `docs/defensa/guion_presentacion.md` | Cumplido (preparado); la exposición la realiza el equipo |
| R26 | Todos los integrantes exponen (asignación aleatoria) | F3 p. 23; F4 p. 22 | OBL (guías) | `docs/defensa/guia_estudio_comun.md`, `docs/defensa/preguntas_defensa.md` (24 preguntas), `docs/defensa/guion_demo.md` | Preparado; la preparación individual y la asignación real son del equipo |
| R27 | Entrega por el líder, carátula con participantes activos | F1 | OBL | Portada del informe | **Pendiente**: nombres `[POR COMPLETAR]` |
| R28 | Rúbrica: Información, Análisis, Conclusiones, | F1; F3 p. 24; F4 | OBL | `reports/informe/informe_PSMA00611.d` | Preparado; falta completar la carátula |

| ID | Requisito | Documento y página | Tipo | Archivo(s) | Estado |
|---|---|---|---|---|---|
| | Estilo/Gramática (0-5 c/u) | p. 23 | | `ocx` y `.pdf` (16 secciones), autoevaluación en el informe | |
| R29 | Mínimo de páginas del informe | Mencionado en la rúbrica; no especificado en F1-F4 (D12) | OBL (indefinido) | — | **Pendiente** de confirmar con el docente; propuesta editorial: 30-40 páginas de cuerpo |
| R30 | Fecha de entrega | F1: 16-nov-2026 23:59 (discrepancias de semana D1-D2) | OBL | — | Registrado |
| M01 | Al menos 15 consultas documentadas (se entregan 20) | Propuesta del equipo | MEJ | `sql/consultas/`, `docs/07_consultas/` | Cumplido (contenedor) |
| M02 | Python con pyodbc y pandas: parámetros enlazados, configuración externa, CSV y gráficos | Propuesta del equipo | MEJ | `src/aerolinea/` | Cumplido (contenedor) |
| M03 | Control de concurrencia probado con sesiones simultáneas | Propuesta del equipo | MEJ | `docs/03_arquitectura/concurrencia.md` | Cumplido (contenedor) |
| M04 | Datos sintéticos deterministas (semilla 20261106) | Propuesta del equipo | MEJ | `generador.py`, `data/sintetico/manifiesto.json` | Cumplido (contenedor) |
| M05 | Seguridad: roles, privilegio mínimo, enmascaramiento, cifrado | Propuesta del equipo | MEJ | `sql/08_seguridad.sql`, `docs/03_arquitectura/seguridad.md` | Cumplido (contenedor) |
| M06 | Pruebas automatizadas (pytest + T-SQL) | Propuesta del equipo | MEJ | `tests/`, `sql/12_pruebas_integridad.sql` | Cumplido (contenedor) |
| M07 | Diccionario y diagramas físicos generados del catálogo | Propuesta del equipo | MEJ | `src/aerolinea/documentar.py` | Cumplido (contenedor) |
| M08 | Zonas horarias y llegadas al día siguiente | Propuesta del equipo | MEJ | `ops.vuelo` (`datetimeoffset`), `rpt.v_vuelo` | Cumplido (contenedor) |
| M09 | Reproducción completa con un comando | Propuesta del equipo | MEJ | `python -m aerolinea reproducir` | Cumplido (contenedor); pendiente en laptop |
| M10 | Referencias en APA 7 verificadas | Propuesta del equipo | MEJ | Informe, sección de referencias; `docs/00_fuentes/revision_fuentes.md` | Cumplido |

*Fuente:* `docs/00_fuentes/matriz_requisitos.md`. *F1: enunciado del aula virtual; F3: guía de aprendizaje general; F4: guía «VIERNES».*

## Anexo B. Hallazgos corregidos durante el desarrollo

**Tabla 20. Defectos detectados y corregidos durante el desarrollo**

| ID | Síntoma observado | Causa | Corrección |
|---|---|---|---|
| H-01 | La carga de 320 000 tiquetes no terminaba tras más de 10 minutos | El plan de las sentencias de los triggers, compilado para una inserción de pocas filas (sin `deleted`), se reutilizaba en actualizaciones masivas | `OPTION (RECOMPILE)` en las sentencias que leen `inserted`/`deleted` |
| H-02 | Tras H-01, la validación de | `SELECT TOP (1)` hace que el | `USE HINT` |

| ID | Síntoma observado | Causa | Corrección |
|---|---|---|---|
| | inmutabilidad (RN-19) seguía siendo lenta | optimizador suponga que la primera coincidencia aparece enseguida (*row goal*) y elija bucles anidados sobre cientos de miles de filas | `('DISABLE_OPTIMIZER_ROWGOAL')` |
| H-03 | Validación de solapes de avión y tripulación cuadrática y con bloqueos de rango amplios | Comparación de cada vuelo con todos los del recurso ($O(n^2)$) con `HOLDLOCK` | Agenda ordenada por recurso con `LAG` ($O(n \log n)$) y bloqueo `UPDLOCK` de la fila padre (`ops.avion`, `rh.empleado`) |
| H-04 | Un empleado sin categoría de tripulación pudo asignarse como capitán | Lógica de tres valores: `NOT (categoria = 'TECNICA')` es *desconocido* cuando la categoría es `NULL`, y la fila no se rechazaba | `ISNULL(c.categoria_tripulacion, '')` en la comparación |
| H-05 | PC-04: solo 6 de 30 asignaciones automáticas simultáneas de asiento tuvieron éxito | Todas las sesiones elegían a la vez «el primer asiento libre» (efecto manada) y chocaban en el índice único | Tomar primero el semáforo `ops.vuelo_cabina` con `UPDLOCK` en `ven.usp_asignar_asiento` |
| H-06 | Error 5074 al aplicar enmascaramiento al número de documento | No se puede alterar una columna con un índice dependiente | Eliminar el índice, enmascarar y recrearlo, dentro de una guarda idempotente |
| H-07 | Error 2628 («los datos se truncarían») al registrar `NO_PRESENTADO` | Columnas de estado `VARCHAR(12)` y el código tiene 13 caracteres | `VARCHAR(15)` en códigos de estado |
| H-08 | El generador fallaba al elegir vendedor en una oficina | Los dos agentes dados de baja eran de la misma oficina y no quedaba ninguno vigente en algunas fechas | Bajas en oficinas distintas; además se corrigieron personas huérfanas tras reservas fallidas, pagos parciales posteriores al plazo, el orden vencimiento/cancelación de vuelo, demoras sin hora estimada y una búsqueda cuadrática de vendedores |
| H-09 | pyodbc no convertía `datetimeoffset` (tipo ODBC -155) | Tipo específico de SQL Server sin conversión nativa | Convertidor de salida en `db.py` |
| H-10 | Un error en la segunda sentencia de un lote no se notificaba | pyodbc devuelve el primer resultado; los errores posteriores aparecen al avanzar con `nextset()` | `consumir_resultados` recorre todos los conjuntos de resultados |
| H-11 | La prueba de permisos del analista pasaba aunque el usuario no tuviera permiso | Cadena de propiedad: dentro de un procedimiento de `dbo`, el acceso a tablas de `dbo` no se vuelve a comprobar | La comprobación se hace con `sp_executesql` (rompe la cadena) bajo `EXECUTE AS USER` |
| H-12 | Conexión rechazada sin configuración explícita (entorno) | ODBC Driver 18 cifra por defecto y valida el certificado; la instancia local usa un certificado autofirmado | Local: `TrustServerCertificate=yes` documentado; Azure: validación obligatoria (`config.py` rechaza lo contrario) |
| H-13 | La carga quedaba **sin confirmar** y bloqueaba las pruebas, aunque el procedimiento terminaba sin error (entorno: Linux/unixODBC) | `pyodbc.pooling` está activo por defecto; unixODBC reutilizaba la sesión física sin restablecerla. La conexión de la carga heredó una transacción implícita abierta de la conexión anterior (`autocommit=False`), así que el `COMMIT` del procedimiento solo cerró el nivel anidado | `pyodbc.pooling = False` y comprobación de `@@TRANCOUNT` antes y después de aplicar la carga |
| H-14 | PR-11: el primer caso se rechazaba por una `CHECK` y no por el trigger que se quería probar | El caso de prueba no aislaba la regla | Se fija `fecha_emision_utc` para que solo falle la transición (50042) |

| ID | Síntoma observado | Causa | Corrección |
|---|---|---|---|
| H-15 | PR-15: el cambio a `EN_VUELO` fallaba por la `CHECK` de salida real | El valor por defecto (hora actual) quedaba fuera de la ventana permitida respecto de la salida programada del escenario | El procedimiento recibe `@momento` explícito en la prueba |
| H-16 | Q18 mostró pilotos con 123 h de vuelo de media y 188 h de máximo en 28 días con los primeros datos sintéticos | Planificación sintética con dos equipos por línea; superaba la referencia de 100 h en 28 días de 14 CFR 117.23. Un primer ajuste a cuatro equipos fijos por línea aún dejó 4 capitanes y 4 primeros oficiales con 105,8 h en las líneas de mayor carga | Equipos por línea calculados con la cota de peor caso (4 o 5 equipos en bloques de 3 días según las horas de bloque de cada línea) y 4 auxiliares de vuelo más en la plantilla sintética |
| H-17 | Tras corregir H-16, PR-15 falló con «no se puede insertar NULL en id_empleado» | La prueba elegía «un capitán sin ninguna asignación»; al usar los 52 capitanes en la rotación ya no quedaba ninguno | La prueba elige tripulantes vigentes sin vuelos en la ventana del escenario (±1 día) y falla con un mensaje explícito si no los hay |
| H-18 | Al ejecutar `11_dml_seguro.sql`, dos ejemplos fallaron en compilación (errores 334 y 10705) | `OUTPUT` sin `INTO` no se admite sobre tablas con triggers habilitados, y `OUTPUT` no admite subconsultas | `OUTPUT … INTO` una variable de tabla y consulta posterior |
| H-19 | La demostración mostraba el aviso 8153 en vez del error de sobrepago y seguía operando sobre un tiquete que ya no existía | (a) `SUM(CASE … END)` sin `ELSE` en `ven.usp_registrar_pago` emitía el aviso 8153 antes del error y pyodbc solo informa el primer registro de diagnóstico; (b) con `XACT_ABORT ON`, el `THROW` del procedimiento revierte **toda** la transacción del llamador, comportamiento correcto que la demostración no contemplaba | `ELSE 0` en la suma; `traducir_error` ignora avisos; la demostración se dividió en dos transacciones (A: atomicidad, B: flujo completo) |
| H-20 | Al repetir las consultas con nivel de compatibilidad 130, Q19 devolvió las mismas filas en otro orden | Un `UNION ALL` sin `ORDER BY` no garantiza el orden; el plan cambió con el nivel de compatibilidad | `ORDER BY control` al final de Q19 |

*Fuente:* `docs/06_pruebas/hallazgos_y_correcciones.md` *(incluye la evidencia de cada corrección).*

## Anexo C. Modelo físico del área de ventas

Diagrama generado desde el catálogo del motor. Los diagramas de las demás áreas están en `docs/02_modelo/diagramas/` (formatos PNG y SVG).

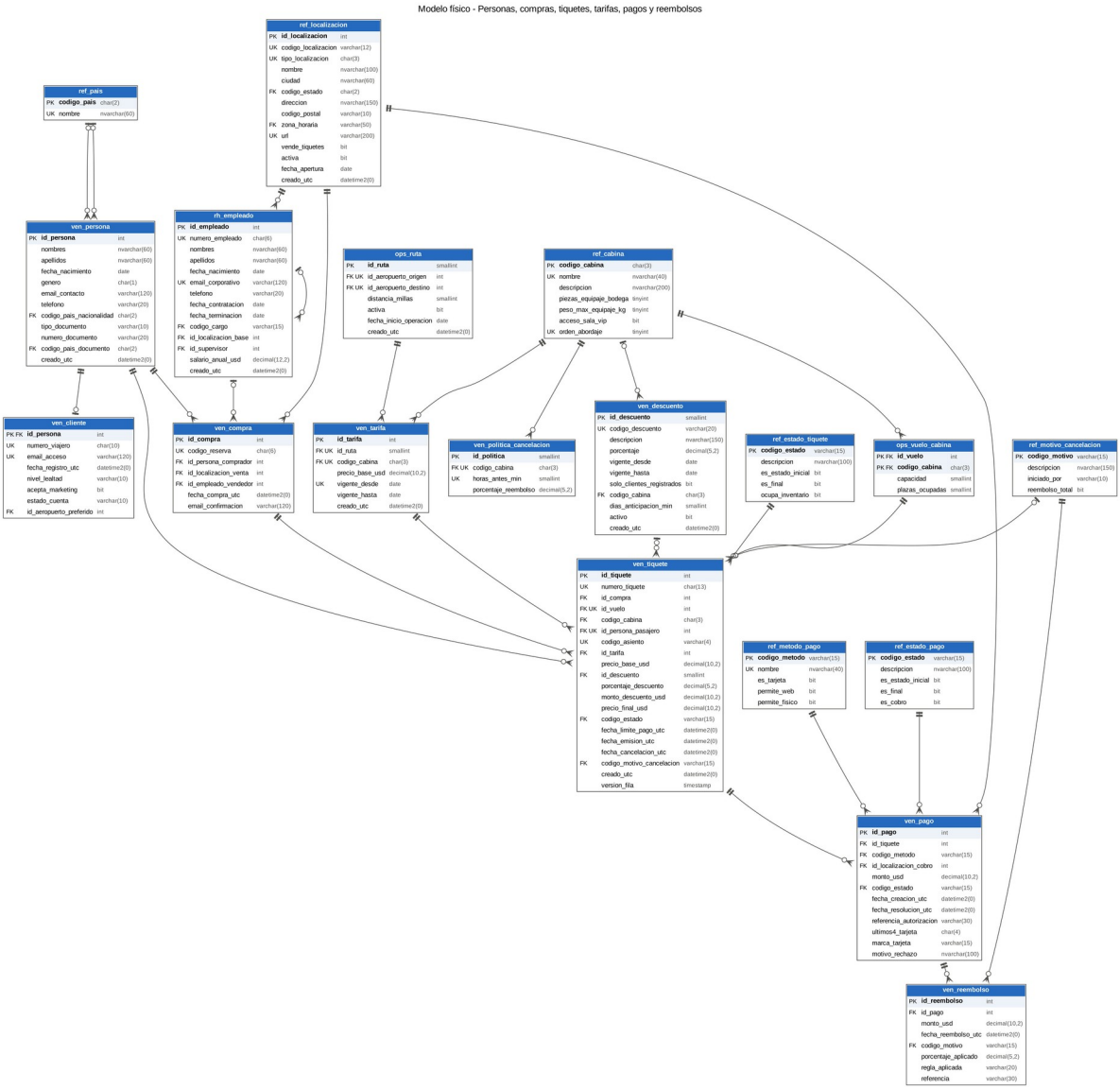



*Figura 15. Modelo físico: personas, compras, tiquetes, tarifas, pagos y reembolsos. Fuente: `docs/02_modelo/diagramas/fisico_ventas.mmd`.*

## Anexo D. Reproducción y archivos

**Tabla 21. Reproducción y ubicación de los archivos**

| Paso | Comando o archivo |
|---|---|
| Verificar el entorno (solo lectura) | `scripts/windows/verificar_entorno.ps1` |
| Instalar dependencias | `pip install -r requirements.txt` |
| Reproducir todo (base local nueva) | `python -m aerolinea reproducir --confirmar-destruccion` |
| Scripts T-SQL y orden | `sql/README.md` |
| Consultas, resultados e interpretaciones | `sql/consultas/`, `reports/resultados/`, `docs/07_consultas/catalogo_consultas.md` |
| Pruebas y evidencias | `docs/06_pruebas/plan_pruebas.md`, `evidence/contenedor_linux/` |
| Despliegue en Azure | `docs/05_azure/guia_azure.md` |
| Este informe | `reports/informe/informe_fuente.md` → |

| Paso | Comando o archivo |
|---|---|
| | `python -m aerolinea informe` |

## Anexo E. Declaración sobre el uso de herramientas y datos

Los datos del proyecto son sintéticos y no contienen información de personas reales. El equipo debe declarar, conforme a las normas de la universidad y del docente, el uso de herramientas de inteligencia artificial en la elaboración del producto y cómo verificó sus resultados: [POR COMPLETAR por el equipo].